THÈSE

Pour obtenir le grade de

DOCTEUR DE L'UNIVERSITÉ GRENOBLE ALPES

École doctorale : EEATS - Electronique, Electrotechnique, Automatique, Traitement du Signal
Spécialité : Nano électronique et Nano technologies
Unité de recherche : CEA /LIST - Laboratoire d'intégration de systèmes et de technologies

Conception d'un Système Mémoire pour Traitement de Données Creuses

Design of a Memory System for Sparse Data Processing

Présentée par :

Valentin ISAAC--CHASSANDE

Direction de thèse :

Frédéric ROUSSEAU PROFESSEUR DES UNIVERSITES, Grenoble INP - UGA	Directeur de thèse
Adrian EVANS CEA	Co-encadrant de thèse

Rapporteurs :

Thèse soutenue publiquement le , devant le jury composé de :

Valentin ISAAC--CHASSANDE
Université Grenoble Alpes
CEA Grenoble (DRT/LIST/DSCIN/LSTA)
TIMA Laboratory (SLS)
ORCID : 0009-0007-9842-5777
valentin.isaacchassande@univ-grenoble-alpes.fr
valentin.isaacchassande@cea.fr

THESIS

DESIGN OF A MEMORY SUB-SYSTEM FOR SPARSE DATA PROCESSING

September 10, 2025

Thesis Adviser:	Pr. Frédéric ROUSSEAU
Thesis Co-Supervisors:	Dr. Adrian EVANS
	Dr. Yves DURAND
CSI Members:	Pr. Yves ROBERT (HDR)
	Pr. Christophe PICARD

EEATS Doctoral College, NENT Speciality
2022 – 2025 Academic Years
From 10.25.2022 to 10.24.2025

Acknowledgements

TODO

TODO *Quote here. Quote here. Quote here. Quote here. Quote here. Quote here. Quote here. Quote here. Quote here. Quote here.*

Author, "Title".

Résumé

TODO

Abstract

TODO

List of Publications

This thesis is based on the following publications:

I Valentin Isaac--Chassande, Adrian Evans, Yves Durand, and Frédéric Rousseau. 2024. Dedicated Hardware Accelerators for Processing of Sparse Matrices and Vectors: A Survey. *ACM Trans. Archit. Code Optim.* 21, 2, Article 27 (Feb. 2024), 26 pages. Reference [44]

II Valentin Isaac--Chassande, Adrian Evans, Yves Durand, and Frédéric Rousseau. 2024. SpDCache: Region-Based Reduction Cache for Outer-Product Sparse Matrix Kernels. In *2024 IEEE 35th International Conference on Application-specific Systems, Architectures and Processors (ASAP)*. IEEE Computer Society, Los Alamitos, CA, USA, 3–7. Reference [45]

III **TODO**

The papers will be referred to in the thesis using their Roman numerals.

Summary

Notations

$$\forall (a,b) \in \mathbb{Z}^2, [\![a,b]\!] = [a,b] \cap \mathbb{Z}$$

nnz, M, A, b and c, $\overline{nnz_r}$, d_r ...

TODO

Glossary

band Refers to an area of a matrix that is concentrated around the diagonal. 9, 16, 17, 18

banded Refers to a matrix structure where the non-zero elements are concentrated around the diagonal. 16, 17, 28, 55

bandwidth With this typography, refers to the size of the band of a matrix. 16

common rank Refers to a rank that is common between two data-entries, when computing a kernel. 18, 19, 23, 25

dense format Is the uncompressed format where all zeros and non-zeros are stored, without any use of metadata. 17, 22, 24

fiber Is the generic term to describe a one-dimensional stream of data or metadata. It corresponds to a row or column in the case of a two-dimensional matrix. A stream of indices is a *fiber* of metadata. 22, 25

gather Refers to reading data from memory. We specifically use this term to categorize data transfers that are irregular. 23, 25, 28

intersection Refers to the process of finding non-zero partial sums, when computing a matrix multiplication. 19, 21, 22, 23, 25, 27, 28, 57

irregular access See "random" access. 9, 15, 28

irregular data Refers to data that has low spatial and temporal locality, thus being more likely to generate irregular accesses. 28

metadata Is data that provides information about other data, such as index arrays of sparse formats. 9, 10, 17, 18, 21, 22, 23, 50

partial sum Refers to the intermediary result before updating the output, when computing a matrix multiplication. 9, 19, 22, 23

'perfectly' banded Refers to a matrix structure where all the non-zero elements are concentrated within a delimitated area around the diagonal (the band). 16

"random" access Is a memory access that occurs on an arbitrary memory address (not necessarily conscutive in memory with previous and next accesses). 9, 15

"random" update Is a "random" access where the access is a read, modify and write on a given address. It is similar to a "random" *reduction*. 23

rank Refers to a dimension of the kernel. 9, 17, 18, 19, 20, 21, 25

reduction Refers to the process of updating an entry. 9, 19, 23, 24, 25, 27, 28, 57

scatter Refers to updating data in memory. We specifically use this term to categorize data transfers that are irregular. 23, 25, 28

sequential accesses Are a group of memory accesses (from a data array, for example) that occur on conscutive memory addresses, in order through time. 17

sparse format Is a compression format for sparse matrices, which avoids storing zeros by using metadata. 9, 17, 18, 20, 21, 22, 23, 28, 50

tile Refers to a sub-region of a *tiling* process. 20, 21, 27

tiling Is a method consisting in splitting data into several independent sub-regions to be processed separately. 10, 20, 21, 23, 25, 27, 28, 50, 55, 57

Acronyms

BaCSC *Banded Compressed Sparse Column*. 17, 18, 28, 55

BCSR *Blocked Compressed Sparse Row*. 17

CNN *Convolutional Neural Network*. 16

COO *COOrdinate*. 17, 18, 24, 28, 50, 55

CSC *Compressed Sparse Column*. 17, 18, 22, 23, 24, 25, 28, 50, 55, 57

CSR *Compressed Sparse Row*. 17, 18, 21, 22, 23, 24, 25, 28, 50, 55, 57

DIA *DIAgonal*. 17

ELL *ELLPACK*. 17

Gust *Gustavson*. 18, 19, 20, 21, 23, 25, 26, 27, 28, 56, 57

HYB *HYBbrid*. 17

IP *Inner-Product*. 18, 19, 20, 21, 22, 23, 25, 26, 27, 28, 56, 57

MM Matrix-Matrix. 18, 19, 20, 25, 28, 57

MV Matrix-Vector. 18, 19, 20, 21, 25, 28, 57

OP *Outer-Product*. 18, 19, 20, 21, 23, 24, 25, 26, 27, 28, 56, 57

PO *partial output*. 19, 20, 21, 23, 25, 28

SpGEMM General Sparse Matrix-Matrix multiplication. 18

SpGEMV General Sparse Matrix-Vector multiplication. 18

SpMM Sparse Matrix-Matrix multiplication. 18

SpMSpM Sparse Matrix-Sparse Matrix multiplication. 18, 20, 22, 23, 24, 25, 26, 27, 28, 56, 57

SpMSpV Sparse Matrix-Sparse Vector multiplication. 18

SpMV Sparse Matrix-Vector multiplication. 12, 18, 21, 28, 55

Introduction

From SoA paper

The execution time of modern scientific and engineering applications is dominated by the manipulation of large sparse matrices, *i.e.* matrices whose number of non-zero entries NNZ is much smaller than their dimensions $M \times N$. Matrices derived from computational physics problems are often sparse and regular because they derive from a relatively small number of differential terms. Algebraic graph problems produce matrices which are still sparse but usually less regular [17]. Similarly, the weight and activation matrices used by Convolutional Neural Networks (CNN) may hold millions of terms, but most of them are explicitly set to zero (*e.g. rectified*) without loss of information [30]. It was recognized early in the 1960s [76] that working on these sparse data sets using dense array representations was inefficient. The seminal work of OSKI [90] in 2003 formalized the sparse representation alternatives and identified many optimizations. This opened the path for specialized software libraries for sparse algebra [94]. However, the single-core performance of software implementations can be deceiving, often only reaching 10% of peak performance [30].

The main matrix multiplication kernels that are of interest are matrix-vector (MV) and matrix-matrix (MM), which include SpMV and SpMSpV ($A \times b = c$), SpGEMV ($A \times b + d = c$), SpMM and SpMSpM ($A \times B = C$), and SpGEMM ($A \times B + D = C$), with *Sp* for *sparse* and *GE* for *general*. SpMV, as well as its variant SpGEMV, are by far the dominant operations of modern algebraic kernels (linear solvers, eigensolvers) which have been explicitly based on them [27, 34, 39, 46, 51, 72, 98]. Since these kernels are ubiquitous, their efficient implementation is primordial. In contrast, the SpMSpM multiplication is less frequent in standard algebraic kernels, but it is extensively used in algebraic graph manipulation [4, 15, 19, 25, 33, 48, 73], as well as in multigrid solvers [3, 14, 55, 75]. The growing importance of large graphs has motivated new research on optimizing SpMSpM applications.

The two matrix operations, SpMV and SpMSpM, pose orthogonal computational challenges: (1) access to sparse input data, (2) *reduction* operations on the intermediate result and (3) formatting and writing back the result to memory. The different algorithms, which are detailed in Section **??**, are built on different articulations of these three aspects. The sparse data storage scheme used in memory dictates the first aspect and is briefly explored in Section **??**. The third aspect, *i.e.* result storage, is sometimes overlooked when the output is dense and not too large. For example, when running on a conventional CPU which is memory-bound, it becomes crucial to streamline the write accesses in order to optimize cache usage when writing a large dense vector output. The second aspect, *i.e.* the *reduction* operations, deserves different solutions according to the execution platform. On a conventional CPU, the bottleneck is memory throughput. Thus, most software implementations of SpMV deal with *a priori* reorganization of the input matrices, by row ordering, blocking and loop optimization, for improving the data locality of operations. The perspective for SpMV is different when dealing with GPU platforms. The challenge shifts to finding a balanced assignment of the numerous threads and warps, and to the use of local memory resources [59]. SpMSpM is far more complex: the intermediate *reductions* introduce new NNZ values at random locations, which trigger costly memory allocations and list management operations [31]. As a consequence, this phase dominates the execution time and becomes the main memory bottleneck on a regular CPU. The issue is similar on GPUs because of the limited size of the scratchpad memory for processing large matrices, which requires complex memory management and load balancing [24, 68, 99].

The main motivation to use custom hardware accelerators is to make better use of the intermediate memory hierarchy, ensuring the input data is available near the cores and also

off-loading the accumulation and *reduction* operations from the processors. The focus of this paper is to review these architectures, to compare them and to analyze their performance. Our main focus is on accelerators that eventually target custom or ASIC implementations, although we do touch on the contributions from accelerators that were prototyped on FPGAs. The reader is referred to [64] for a survey of acceleration techniques for CPUs, GPUs and FPGAs. Section **??** reviews the storage schemes and the three basic algorithms with a focus on the data access patterns and their impact on the memory hierarchy. In Section **??** we study how the different algorithms stress the memory systems and propose an analytical model based on these insights. In Section 2.2 we present the operation of several state-of-the-art hardware accelerators, in the light of these hardware challenges. In Section 2.3, we systematically compare these accelerators. Throughout this study, we try to be consistent, and we hope that our nomenclature will be useful for architects and designers who intend to implement dedicated hardware for problems with sparse datasets.

THE rapid growth of data-intensive applications has made sparse and irregular data structures increasingly prevalent in scientific computing, machine learning, and graph analytics. However, efficiently processing sparse matrices remains a fundamental challenge due to their inherent irregularity, which leads to unpredictable memory access patterns, limited opportunities for data reuse, and suboptimal utilization of hardware resources. Existing hardware accelerators have demonstrated partial solutions, yet they often rely on specialized assumptions, exhibit limited scalability, or fail to fully exploit reduction and memory access locality.

This thesis addresses the following central question:

PROBLEMATIC

How can hardware and architectural mechanisms be designed to efficiently support the computation of irregular sparse data, in particular random reductions, while ensuring scalability, adaptability, and performance across diverse workloads?

To tackle this problem, we investigate the limitations of existing algorithms and accelerators, propose a novel cache architecture with reduction support tailored to sparse data, and evaluate its behavior through modeling, simulation, and hardware prototyping.

This manuscript is structured around several key contributions. We begin with an in-depth discussion of irregular data, with a particular emphasis on sparse matrices (Chapter 1). This includes a study of fundamental sparse matrix multiplication kernels and their associated algorithms, highlighting the main hardware challenges that arise when dealing with sparse data. A high-level analysis of algorithms for sparse data computing is provided, with a focus on basic storage schemes and opportunities for data reuse. Building on this foundation, we review the state-of-the-art in hardware accelerators designed for efficient sparse matrix computing (Chapter 2). This review identifies common characteristics, proposes a classification of existing accelerators and design tendencies, and offers a comparative analysis to better understand the rationale behind specific architectural choices under different objectives and constraints.

Following this survey, we introduce a novel cache architecture specifically designed to address the challenge of "random" reductions (Chapter 3). The proposed cache integrates reduction support and organizes its memory into multiple regions tailored to denser and sparser data patterns. A Python model was developed to simulate memory transactions in this cache and to compare its behavior against that of a generic cache. To further investigate its performance, we complement this analysis with a probabilistic model, implemented in C, that enables rapid simulation and design space exploration (Chapter 4). At the micro-architectural level, we detail the virtual partitioning of cache memory into multiple configurable regions, which ensures both flexibility and adaptability to diverse workloads (Chapter 5).

The proposed cache is then evaluated through RTL simulation, which reveals two distinct groups of matrices that exhibit different performance profiles (Chapter 6). Finally, the

manuscript outlines potential future enhancements to improve both performance and scalability, including a multicore extension of the cache architecture and optimizations targeting bandwidth efficiency (Chapter 7).

Contributions of the Thesis

The main contributions of this thesis can be summarized as follows:

1. **Analysis of irregular and sparse data computing** (Chapter 1)
   - Study of fundamental sparse matrix multiplication kernels and algorithms.
   - Identification of the key hardware challenges associated with sparse data processing.
   - High-level analysis of algorithms for sparse data computing, with emphasis on storage schemes and data reuse opportunities.
2. **Survey and classification of state-of-the-art accelerators** (Chapter 2)
   - Comprehensive review of hardware accelerators for sparse matrix computing.
   - Identification of common architectural features and design tendencies.
   - Comparative analysis to explain design choices under varying objectives and constraints.
3. **Design of a cache architecture for random reductions** (Chapter 3)
   - Proposal of a cache with native reduction support, capable of handling both dense and sparse data via multiple cache regions.
   - Development of a Python-based simulation model to analyze memory transactions and benchmark against a generic cache.
4. **Probabilistic modeling and exploration of cache behavior** (Chapter 4)
   - Implementation of a lightweight C-based probabilistic model for rapid simulation.
   - Exploration of cache dimensioning and behavior across diverse workloads.
5. **Micro-architecture of the reduction cache** (Chapter 5)
   - Introduction of a virtual memory partitioning scheme enabling multiple configurable regions.
   - Emphasis on flexibility and adaptability in cache design.
6. **Evaluation through RTL simulation** (Chapter 6)
   - Assessment of the proposed cache architecture using RTL-level simulation.
   - Identification of two distinct groups of sparse matrices leading to different performance profiles.
7. **Future directions for performance and scalability** (Chapter 7)
   - Proposal of a multicore version of the cache to extend scalability.
   - Exploration of bandwidth optimization strategies.

Chapter 1

Background

Contents

1.1 Introduction

MODERN computing systems are designed to deliver high performance by exploiting memory hierarchies, parallelism, and increasingly sophisticated hardware features. However, the efficiency of these systems is strongly tied to the regularity of memory access patterns. Applications with predictable, sequential data access can fully benefit from cache locality, prefetching mechanisms, and vectorization. In contrast, applications characterized by irregular accesses, where memory locations are not known in advance or follow non-contiguous patterns, pose significant challenges.

Irregular access patterns are common in domains such as graph analytics, sparse linear algebra, unstructured mesh computations, and database queries. In these scenarios, pointer chasing, indirect indexing, and dynamically changing data structures disrupt spatial and temporal locality. As a result, the hardware struggles to hide memory latencies, leading to poor cache utilization and underutilization of available parallelism.

The impact of irregular accesses extends beyond raw performance. They also complicate algorithm design, hinder scalability on many-core architectures, and increase energy consumption due to repeated off-chip memory requests. Understanding and addressing these challenges is therefore a central concern in both system design and application optimization. This chapter provides the necessary background for analyzing irregular accesses, highlighting their sources, effects, and the architectural considerations they expose, focusing on the case of indirect indexing in sparse matrix computing. Most of the information of this chapter are based on already published content from Papers I and II.

DEFINITION

Irregular accesses, or **"random" accesses**, occurs when memory is accessed on non-consecutive, hard-to-predict addresses.

1.2 Sparse Matrices

In most applications, irregular data can be found in matrix computing, where large matrices filled with many zeros are stored in compressed format in memory. These sparse matrices are commonly used in diverse fields such as linear algebra, where sparse matrix multiplication is used to solve linear systems of equations for scientific purposes or simulation, as well as graph analytics where the vertices of a graph are represented as a sparse matrix that can be easily analysed with matrix operations [17], or finally *Convolutional Neural Networks* (*CNNs*) or other similar AI features where weights and activations can be represented and processed as sparse matrices [30]. In these contexts, matrices are large, with dimensions up to billions of elements, and highly sparse, with non-zero densities down to 10^{-7} [52].

A matrix is defined as a two dimensional array. We note matrices with capital letters A, B, C, conversely to vectors a, b, c. Their dimension are denoted using M, N and K. For example, we can define a square matrix A of dimension M, or a matrix B of dimension (K, N). We note nnz the number of non-zero elements of a matrix, and d the associated density. With the example of a matrix $A(M, N)$:

$$d_A = \frac{nnz_A}{M \times N}$$

DEFINITION

A **sparse matrix** is a matrix filled with many zeros.

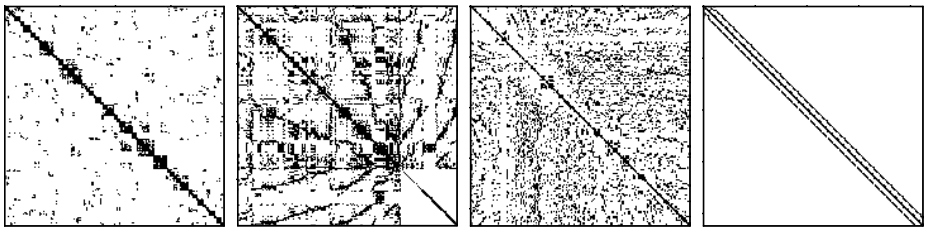
NOTE

In general, we consider a matrix to be highly sparse when its number of non-zero elements has the same order of magnitude than its dimension, $nnz \sim M$. But there is no intrinsic definition of the frontier between a sparse and a dense matrix. In this manuscript, we consider a matrix as sparse when it needs to be compressed in memory (see Section 1.3).

1.2.1 Banded Matrices

Matrices used in scientific computation originate from physical modelling where a system of partial differential equations is discretized, resulting in a large sparse matrix. The regularity of this generation process naturally creates banded matrices. Moreover, numerical techniques exist to transform matrices into banded matrices [61, 62, 92].

A banded matrix is a matrix which has a dense region around its main diagonal and which is sparser away from the diagonal. To give few visual examples, Figure 1.1 shows the structure plots for four typical matrices found in the SuiteSparse Matrix Collection [52], having a high-density band along the diagonal. In Figure 1.2, we illustrate a banded, square matrix of dimension M. We define w_B as the width of the banded region, or *bandwidth*, as shown in *red* in the figure. Sometimes it is easier to refer to the half-*bandwidth*, w_{hB}, which respects the relation $w_B = 2 \times w_{hB} - 1$. The average density in the band and out of the band is defined as d_{IB} and d_{OB}, respectively. As in Figure 1.1d, in a 'perfectly' banded matrix, there are no elements outside the band, thus $d_{OB} = 0$.



(a) *roadNet-CA* **(b)** *scircuit* **(c)** *msc10848* **(d)** *raefsky3*

Figure 1.1: Examples of Banded Matrix Structure

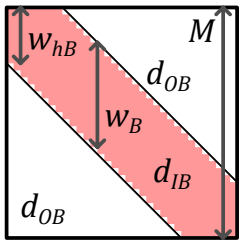


Figure 1.2: Banded Matrix Definition

DEFINITION

A **banded matrix** is a sparse matrix which has a higher density of non-zero elements around the diagonal. A **'perfectly' banded matrix** concentrates all its non-zero elements within a delimitated region around the diagonal. This region is called the band.

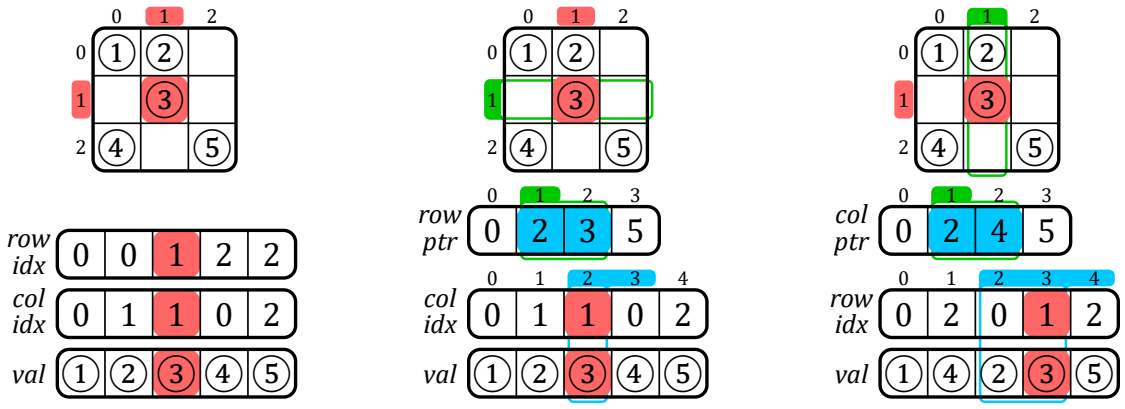
> **NOTE**
>
> The SuiteSparse Matrix Collection [52] is a large, open source data-base of real application sparse matrices coming from various domains. It gathers around 3,000 matrices of various sizes, densities and structure, which are extensively used for benchmarking (see Chapter 2).

1.3 Sparse Formats

In the absence of a compressed format, matrices are stored in fixed size two-dimensional arrays. With this dense format, elements can be randomly accessed ($\mathcal{O}(1)$), and sequential accesses are highly efficient. The two dimensions are called ranks which correspond to the rows and columns. For matrices, there are thus two variants of the dense format, depending on which rank is stored sequentially in memory. In a *row-wise* (or *row-major*) storage, the elements within the rows are sequential, and conversely for *column-wise* (or *column-major*).

Sparse matrices contain a very large number of zeros, and to avoid storing these repeated zeros, there exist many compressed formats [11, 13, 20, 49, 76, 89], where certain formats are best-suited for matrices with a specific structure. The main principle of these sparse formats is to avoid storing the zero elements, compensating with additional metadata which gives the position of the non-zero elements.

We can cite some of the most used sparse formats like *COOrdinate* (COO) that uses three tables of size nnz with only the non-zero elements (val table in Figure 1.3a) and their row and column indices (row and col idx tables). This format is the simplest one. It does not need to be sorted to be effective which is a great advantage when the data is "randomly" updated. However, it is an inconvenient for matrix sequential traversals or search tasks. This format can be sorted *row-wise* as in the example in Figure 1.3a or *column-wise* if needed, but would lose its advantage compared to the two mostly used sparse formats *Compressed Sparse Row* (CSR) (Figure 1.3b) and its *column-major* counterpart *Compressed Sparse Column* (CSC) (Figure 1.3c), which are sorted by construction. With CSC, the val table of size nnz stores the non-zero elements sorted *column-wise* and the idx table of size nnz stores the corresponding row indices, similarly to COO. The third table ptr of size $M + 1$ stores the positions of the columns. For example, in Figure 1.3c, the second column of index 1 (*green*) has non-zero elements between positions 2 and 4 (*blue*, 4 excluded) of tables idx and val. This ptr table allows CSR and CSC to have a lower memory footprint than COO.



(a) COO **(b)** CSR **(c)** CSC

Figure 1.3: Example of a Sparse Matrix in COO, CSR and CSC Formats

Many others classes of sparse formats exists such as *Blocked Compressed Sparse Row* (BCSR), *DIAgonal* (DIA), *ELLPACK* (ELL) and *HYBbrid* (HYB), which all have their own advantages depending on the matrix structure or the hardware setup. Some details on these formats are mentioned in Appendix A.

In general, there is many derivates from the above main categories of sparse formats. Ultimately, we can create any format we want depending of our needs and constraints in software and hardware. Latter in this manuscript, we indeed use a custom sparse format called *Banded Compressed Sparse Column* (BaCSC) that is an extension of CSC for banded matrices. As shown in Figure 1.4, we added extra information in the ptr table to identify the beginning and end of the band. Indeed, three pointers instead of one are now needed per

column, as shown with the example of column 2 in *yellow*. The column is split in one in-band (IB) region in the middle and two out-of-band (OB) regions around. The ptr table size thus becomes $3 \times M + 1$. With this format, we can avoid the computational cost of identifying in which region non-zero elements are, while traversing the matrix, a feature we take advantage of in Chapter 6.

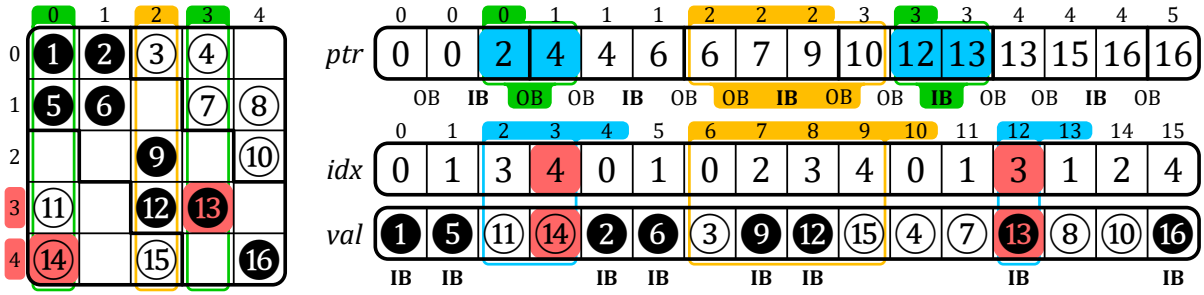


Figure 1.4: Example of a Sparse Matrix in BaCSC Format

DEFINITION

A **sparse format** is a compression format for sparse matrices, which avoids storing zeros by using metadata.

NOTE

The mostly used sparse formats are COO, CSR and CSC.

CONCLUSION

With the use of sparse formats, it is possible to greatly **decrease memory footprints** while increasing the data dimensions. It allows great **flexibility** to manage data structure in software and hardware.

1.4 Algorithms for Dense and Sparse Matrix Multiplication

THE main matrix multiplication kernels that are of interest are Matrix-Vector (MV) and Matrix-Matrix (MM), which include SpMV and SpMSpV ($A \times b = c$), SpGEMV ($A \times b + d = c$), SpMM and SpMSpM ($A \times B = C$), and SpGEMM ($A \times B + D = C$), with 'Sp' for *sparse* and 'GE' for *general*. In the following sections, we focus on SpMV and SpMSpM, with the notation $A(M,K) \times b(K) = c(M)$ and $A(M,K) \times B(K,N) = C(M,N)$, respectively. The rank K is the common rank between the two inputs A and B (or b).

The existence of different ranks allows several algorithmic possibilities to compute matrix multiplication: the *Inner-Product* (`IP`) algorithm, the *Outer-Product* (`OP`) algorithm and *Gustavson*'s (`Gust`) algorithm [28, 37]. In all cases, the matrix A is traversed sequentially. The three algorithms, however, impose different access patterns for B (or b) and C (or c). Indeed, in many cases, they have to be accessed multiple times, as discussed in the following sections.

NOTE

In this manuscript, we focus on only two kernels, SpMV and SpMSpM. Note that, among MV kernels, SpMV is the most used and we specifically work on this kernel in Chapter 6. Similarly, SpMSpM is widely used among MM kernels, and serves as a great case study to understand the limitations of sparse MM computing. Also note that the *general* ('GE') versions of these kernels do not fundamentally affect the hereafter analysis and conclusions.

1.4.0.1 Inner-Product Algorithm

The *Inner-Product* algorithm [28] is an intuitive way to calculate a matrix multiplication because it follows the loop order of the mathematical formulas (1.1) and (1.2) for MV and MM, respectively. The common rank K is the innermost loop of the algorithm (see Algorithms 1.1 and 1.2, lines 2 and 3, respectively). The output C (or c) is always updated sequentially, but the B-matrix (or b-vector) will be entirely read M times. In the context of sparse matrices,

the reads of B (or b) are conditioned by the non-zero elements of A, which requires indirect accesses. If B (or b) is also sparse, the algorithm needs to perform *intersections* between the non-zero elements in the rows of A and the columns of B (or in the b-vector) (see Section 1.5). In Algorithm 1.2, exchanging M and N (lines 1 and 2) simply causes the output to be updated *column-wise*.

$$\forall i \in [\![0, M-1]\!],\ c_i = \sum_{k=0}^{K-1} A_{i,k} \times b_k \tag{1.1}$$

$$\forall (i,j) \in [\![0, M-1]\!] \times [\![0, N-1]\!],\ C_{i,j} = \sum_{k=0}^{K-1} A_{i,k} \times B_{k,j} \tag{1.2}$$

Algorithm 1.1: Dense Matrix-Vector *Inner-Product* Algorithm

```
1 for i ∈ [0, M − 1] do
2 |   for k ∈ [0, K − 1] do
3 |   |   c_i += A_{i,k} × b_k
```

Algorithm 1.2: Dense Matrix-Matrix *Inner-Product* Algorithm

```
1 for i ∈ [0, M − 1] do
2 |   for j ∈ [0, N − 1] do
3 |   |   for k ∈ [0, K − 1] do
4 |   |   |   C_{i,j} += A_{i,k} × B_{k,j}
```

1.4.0.2 Outer-Product Algorithm

The *Outer-Product* algorithm [28] has the opposite rank loop order than the *Inner-Product*. Indeed, Algorithms 1.3 and 1.4 show that the common rank K is the outermost loop (line 1). It changes nothing mathematically except that the B-matrix (or b-vector) is always read sequentially. In the context of sparse matrices, the output C (or c) is updated with indirect accesses. The algorithm needs to perform *reductions* of the partial sums $A_{j,k} \times B_{k,j}$ (or $A_{j,k} \times b_k$), as we show in Section 1.5. To avoid irregularity when updating C (or c), the `OP` algorithm is often separated into two phases. First, during the *multiply* phase, we compute several *partial outputs* (*POs*) consisting of intermediate matrices (or vectors) of partial sums $A_{j,k} \times B_{k,j}$ (or $A_{j,k} \times b_k$), then during the *merge* phase, we accumulate all the *POs* to form the final C-matrix (or c-vector). With this method, `OP` necessitates the generation of at most K *POs* of densities conditioned by the non-zeros elements of both inputs A and B (or b).

Algorithm 1.3: Dense Matrix-Vector *Outer-Product* Algorithm

```
1 for k ∈ [0, K − 1] do
2 |   for i ∈ [0, M − 1] do
3 |   |   c_i += A_{i,k} × b_k
```

Algorithm 1.4: Dense Matrix-Matrix *Outer-Product* Algorithm

```
1 for k ∈ [0, K − 1] do
2 |   for i ∈ [0, M − 1] do
3 |   |   for j ∈ [0, N − 1] do
4 |   |   |   C_{i,j} += A_{i,k} × B_{k,j}
```

1.4.0.3 Gustavson's Algorithm

As the MM algorithms have three ranks instead of two for MV, there is a third ordering, known as *Gustavson*'s algorithm [37] where the common rank K is the middle-loop (see line 2 in Algorithm 1.5). With this approach, the output is computed row-by-row with *PO*-rows generated and accumulated for each output C-row. With this algorithm, updates of the C-rows are sequential, but elements within the rows are updated with indirect accesses. In the dense case, the B-matrix is entirely read M times and K *PO*-rows of size N are generated. In the context of sparse matrices, conversely to the C-rows, the selection of the rows in B is indirect, but the accesses are sequential within a row. Thus, the *intersection* search is only needed for the B-rows and not each element. The *PO*-rows can be accumulated before computing the next C-row. For this algorithm, exchanging M and N causes the three matrices to be accessed *column-wise* instead of *row-wise*.

Algorithm 1.5: Dense Matrix-Matrix *Gustavson*'s Algorithm

```
1 for i ∈ [0, M − 1] do
2 |   for k ∈ [0, K − 1] do
3 |   |   for j ∈ [0, N − 1] do
4 |   |   |   C_{i,j} += A_{i,k} × B_{k,j}
```

1.4.0.4 Algorithms Comparison

In Table 1.1, we summarize the main features of **IP**, **OP** and **Gust** in terms of accesses to the matrix A, B (or b) and C (or c). With this high level analysis, we can easily see that the data that is accessed with indirections shifts depending of the algorithms, from the input B (or b) to the output C (or c), with **Gust** being a trade-off between **IP** and **OP** for MM multiplication. To have a better understanding of the consequences on the actual number of memory accesses, we propose a more detailed analysis in Section 1.5.4, for the case of SpMSpM.

NOTE

Note that these algorithms are available in dense as well as in sparse MV and MM multiplications. In sparse, the above algorithms (from 1.1 to 1.5) have the same behavior from a software point-of-view, but need adjustments to traverse a sparse format instead of a dense matrix. Sparse versions of these algorithms are presented in Section 1.5.

Table 1.1: Comparison of Matrix Multiplication Algorithms

	***Inner-Product* (IP)**[1]	***Outer-Product* (OP)**[1]	***Gustavson* (Gust)**[2]
A(M,K) **Accesses**	• *row-wise* • read N times[3,4] • sequential	• *column-wise* • read once • sequential	• *row-wise* • read once • sequential
B(K,N) **Accesses**	• *column-wise* • read M times[5] • not sequential	• *row-wise*[6] • read $M \times d(A)$ times[4,7] • sequential	• *row-wise* • read $M \times d(A)$ times[7] • sequential within rows
C(M,N) **Accesses**	• generated *row-wise*[6] • one sequential traversal	• generated without specific structure • K *POs* produced[8] (each *PO* has $\overline{nnz_c}(A) \times \overline{nnz_r}(B)$ non-zero elements[9])	• generated *row-wise* • $K \times d(A)$ *PO*-rows produced[10] for each row of A (M) ($\overline{nnz_r}(B)$ non-zero elements per *PO*-row)

[1] Considering $N = 1$ in the case of MV multiplication.
[2] Only for MM multiplication.
[3] Or less than N times if B has empty columns.
[4] Read once in the case of MV multiplication.
[5] If B is stored with a sorted sparse format, else it would be read $\overline{nnz_r}(A) \times M = nnz(A)$ times.
[6] *element-wise* in the case of MV multiplication (there is no row).
[7] Which is equal to $\overline{nnz_c}(A)$, the average number of non-zero elements in the columns of A.
[8] Or less than K times if A has empty columns or B has empty rows.
[9] $\overline{nnz_c}(A)$ in the case of MV multiplication.
[10] Which is equal to $\overline{nnz_r}(A)$, the average number of non-zero elements in the rows of A.

CONCLUSION

There are *three main algorithms* to handle sparse MV and MM kernels, ***Inner-Product* (IP)**, ***Outer-Product* (OP)** and ***Gustavson* (Gust)**. Each presents different characteristics, the main one being the **irregularity of the accesses**: on the **input** matrix B (or vector b) with **IP**; on the **output** matrix C (or vector c) with **OP**; on both with **Gust** but at **row scale**.

1.4.1 Tiling

The *tiling* process [35] consists in matrix partitioning: a matrix may be divided into *tiles*, non-overlapping sub-matrices. In other words, this method consists of adding ranks to delimit the *tiles*, and thus adding loop iterations in the kernel algorithm.

Tiling methods allow to locally reduce the dimensions of data and thus to adapt the working set to the size of the local memory [53, 93], particularly when the matrix has a structure with denser regions. *Tiling* can also increase parallelism opportunities. Software optimizations for sparse matrix computation, such as OSKI [90] and Sparse BLAS [27], are based on *tiling*. In previous sections, the study of the **IP**, **OP** and **Gust** algorithms highlighted the influence of rank ordering on the sequentiality and data movement, and thus, adding intermediate ranks with *tiling* breaks the regularity of pure **IP** or **OP** algorithms, requiring additional hardware support (see Section 1.5).

For example, let us consider the A-matrix in Figure 1.5 which is divided into four *tiles* (one level of *tiling*). For a SpMV, two new ranks appear for a total of four ranks, as shown in Algorithm 1.6: M_1 and K_1 allow *tile* selection and M_2 and K_2 allow data access within a *tile*. Thus, we could decide to apply an **OP** on *tiles* and conversely an **IP** within *tiles*, mixing-up the pros and cons of these two algorithms at different rank levels. In the case of the example in Figure 1.5, the upper-left A-*tile* (*red*) is evaluated first, sequentially updating the upper *tile* of the c-vector in an **IP** manner. The lower-left A-*tile* (*yellow*) is then computed the same way. At this point, *intersections* have been done with only the upper b-*tile*, and c has been updated sequentially (first *PO*). The two next A-*tiles* (*green*, then *blue*) will compute *intersections* with the lower b-*tile* and update the entire c-vector sequentially, showing the **OP** characteristic of having *POs* to *merge*, two in this case.

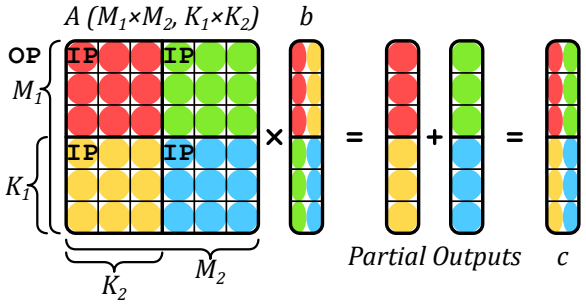


Figure 1.5: An Example of *Tiling* on SpMV

Algorithm 1.6: Matrix-Vector Algorithm with Custom *Tiling* of Figure 1.5

```
1 for k1 ∈ [[0, K1 − 1]] do
2     for i1 ∈ [[0, M1 − 1]] do
3         for i2 ∈ [[0, M2 − 1]] do
4             for k2 ∈ [[0, K2 − 1]] do
5                 i ← i1 × M2 + i2
6                 k ← k1 × K2 + k2
7                 ci += Ai,k × bk
```

DEFINITION

Tiling is a method for partitioning matrices in non-overlapping sub-matrices, called ***tiles***.

NOTE

Note that in case of *tiling*, the A-matrix is no longer read sequentially, which can be problematic if it is stored in a classical sparse format like CSR that is specifically made for *row-wise* accesses. Custom sparse formats adapted for *tiling* can be used [42], but this requires a preprocessing conversion.

CONCLUSION

Tiling methods are widely used to **distribute workloads** across multicore systems, and to **locally reduce the data dimensions**. Complex *tiling* algorithms with many possible rank ordering **inherits the behaviors of IP and OP**, and can be described as a combinations of both at different *tile* levels.

1.5 Hardware Challenges for Sparse Matrix Computation

In the previous section we presented a brief background on matrix multiplication without considering the underlying hardware. Although the various sparse formats reduce the memory footprint, they have the disadvantage that indirect accesses are required, which creates data-dependencies and reduces the effectiveness of caches. As we have seen, with all three algorithms, at least some of the accesses are irregular. The lack of spatial locality for these accesses reduces the benefit of having caches [63, 96]. In other words, a full cache-line is loaded, but only a few entries are read or modified. Moreover, the matrices of interest are extremely large and even the non-zero elements, and associated metadata, can not generally

fit in the cache. Depending on the algorithm, successive accesses to the same element may be too far apart to benefit from temporal locality in the cache.

1.5.1 Inner-Product Algorithm

In an **IP** algorithm, the A-metadata needs to be read to indirectly access the B-matrix (or b-vector), which may itself be stored in a sparse format. In this case, the algorithm needs to check if the non-zero A-indices are present in B (or b). If an index appears in neither list, there is no need to perform the multiplication (*ineffectual operation*). More generally, this problem applies to any *fiber* and is called the *intersection* search. As a minimum, this requires reading all the metadata for A and B (or b), and identifying those present in both.

In Algorithm 1.7, we present the pseudo-code for an *Inner-Product* (**IP**) SpMSpM with A and B stored in CSR and CSC formats, respectively. The outer-loop (M, line 1) iterates over the rows of A. The middle-loop (N, line 2) iterates over the columns of B. The inner-loop (line 4) iterates over the non-zero elements of the current row (i) of A, and for each index (k_A of position pos_A in the idx array of A, line 5), a search is performed in the metadata of B (line 6), to see if the corresponding index in B is present. Assuming it is ($isMatch$ is true, line 7), the partial sum is computed and then accumulated locally (acc, line 8). This example highlights the need to efficiently search for the *intersection* of indices in the metadata of the inputs.

Different algorithms can be used to address the *intersection* search, depending on whether the elements are sorted. When the elements are sorted, the *intersection* becomes a *merge* and generally requires at most $2 \times nnz - 1$ comparisons (in some cases the number of comparisons can be reduced [7]). In Algorithm 1.7, we use sorted sparse formats (CSR and CSC). Thus, the naïve *intersection* search proposed (line 10) does at most one traversal of each the current row of A (i) and the current column of B (j). Note that computing an *intersection* in software induces the use of conditional loops and branches, which can be costly for classical CPU branch predictors. As we see in Chapter 2, certain accelerators provide support for performing this *intersection* operation directly in hardware.

Algorithm 1.7: **IP** SpMSpM Performing *Intersection* Searches with CSR and CSC Formats

Data: $A(M, K)$ in CSR: $A.val, A.idx, A.ptr$
Data: $B(K, N)$ in CSC: $B.val, B.idx, B.ptr$
Data: $C(M, N)$ in dense format (for readability)
Result: $C = A \times B$

1 **for** $i \in [\![0, M-1]\!]$ **do** // Iterations over the rows of A
2 **for** $j \in [\![0, N-1]\!]$ **do** // Iterations over the columns of B
3 $acc \leftarrow 0$; $pos_B \leftarrow B.ptr_j$;
4 **for** $pos_A \in [\![A.ptr_i, A.ptr_{i+1} - 1]\!]$ **do**
 // Iterations over the non-zero elements of the i-th row of A
5 $k_A \leftarrow A.idx_{pos_A}$;
 // Column-index of the (i, pos_A) non-zero element of A
6 $(isMatch, pos_B) \leftarrow IntersectionSearch(k_A, j, pos_B, B.idx, B.ptr)$;
 // Return true and the matching pos_B if idx exists in the j-th column of B
7 **if** $isMatch$ **then** // If indices did match
8 $acc \leftarrow acc + A.val_{pos_A} \times B.val_{pos_B}$;
 // Local partial sum accumulation
9 $C_{i,j} \leftarrow acc$; // Write in memory of the resulting (i, j) element of C

10 **function** $IntersectionSearch(k_A, j, pos_B, B.idx, B.ptr)$ **is**
  // Naive intersection search allowing the i-th row of A and the j-th column of B to be traversed only once at iteration (i, j)
11 **while** $B.idx_{pos_B} < k_A$ ***and*** $pos_B < B.ptr_{j+1}$ **do**
12 $pos_B \leftarrow pos_B + 1$;
13 **if** $B.idx_{pos_B} = k_A$ **then** // Returns true if indices from A and B match
14 **return** $(true, pos_B)$;
15 **return** $(false, pos_B)$;

DEFINITION

The ***intersection* search** consists in founding non-zero elements of two sparse arrays that have matching indices. This search is costly in software on classical CPUs, thus being key to improve performance of **IP** algorithm.

1.5.2 Outer-Product Algorithm

In an **OP** algorithm, the inputs A and B (or b) are accessed sequentially, but the updates of the output C (or c) require indirect accesses. The pattern of the accesses, while the output is generated, is irregular and determined by the structure of the inputs. Furthermore, if these "random" updates provoke cache misses where only one word within a cache-line is modified, the bandwidth to external memory is poorly utilized. The **OP** algorithm generates several *partial outputs* (*POs*) that must be accumulated to obtain the final output. We can identify two strategies: *immediate update* and *deferred update*. In the former, the final output is immediately updated as each partial sum is generated. If the output is stored using a sparse format, either the index being updated does not yet exist, and the new partial sum must be inserted. Or, if it exists, a lookup must be done to locate it, the update performed and the result written back. The problem associated with combining the *POs* is called *reduction*.

In Algorithm 1.8, we present two pseudo-codes for an *Outer-Product* (**OP**) SpMSpM using *immediate update* (from line 1) and *deferred update* (from line 7) with A and B stored in CSC and CSR formats, respectively. The outer-loop (K, line 1 or 7) iterates over the columns of A and the rows of B (common rank). The middle-loop (line 2 or 9) iterates over the non-zero elements of the current column (k) of A. The inner-loop (line 4 or 11) iterates over the non-zero elements of the current row (k) of B. Because we only iterate over the non-zero values, there is no need for an *intersection* search. The partial sum is then computed (line 6 or 15). Finally, in the *immediate update* version, the partial sum must be accumulated to C (line 6). These accesses are irregular, making it necessary to lookup the value in C, update it, and write it back. This is a *reduction* operation, performed on irregular data.

To avoid the complexities of these "random" *reductions* with *immediate updates*, the *deferred update* version postpones the accumulation and simply stores the partial sums in memory. One such technique is called *output batching* [50]. With this technique, arrays containing the indices and the partial sum are streamed sequentially to memory (PO_k, lines 13-15). Then, during a second pass (line 17), they are read back and the accumulations are performed, an approach which results in sequential memory accesses during both the first and second passes, with the downside that the volume of data transferred to memory is increased. As we see in Chapter 2, hardware accelerators may combine the *immediate* and *deferred update* techniques to optimize memory bandwidth.

DEFINITION

***"Random"* reductions** occur when updating data from memory on irregular addresses, which happen on *immediate update* **OP**. With *deferred update* **OP**, the *reductions* are sequential but large sets of data need to be buffered in memory. Optimizing *reduction* operations is key to improve performance on **OP** algorithm.

1.5.3 Summary

In Table 1.2, we summarize the previously discussed advantages and hardware challenges for each algorithm, including **Gust** which presents a trade-off between **IP** and **OP**. The potential for parallelism and data-reuse varies across algorithms, **IP** being the most limited unless *tiling* methods are used. With **OP**, *POs* can easily be generated across a multicore system but at the cost of transferring many data to memory. With *immediate updates*, **OP** transfers less data but all accesses on the output are "random" *reductions*.

The issues presented with **IP** and **OP** can be generalized to two concepts: *gather* and *scatter*. The *gather* problem exists when the inputs are indirectly accessed, and need to be read from memory using metadata to present the processor with the correct terms for computing the partial sums. Typically, with **IP**, the *gather* problem is more difficult. The *scatter* problem exists when the order in which the output matrix/vector is generated is not sequential and is typically associated with the **OP** algorithm. In the case of *tiling*, a hardware accelerator must handle both *gather* and *scatter*, creating a need for hardware support for both *intersection* searches and *reductions*.

Algorithm 1.8: OP SpMSpM Performing *Reduction* with CSR and CSC Formats

Data: $A(M,K)$ in CSR: $A.val, A.idx, A.ptr$
Data: $B(K,N)$ in CSC: $B.val, B.idx, B.ptr$
Data: $PO_k(M,N)$ for all $k \in [\![0, K-1]\!]$ in COO: $PO_k.val, PO_k.row, PO_k.col$
Data: $C(M,N)$ in dense format (for readability)
Result: $C = A \times B$

// *
// * * * 'Immediate updates' version *
for $k \in [\![0, K-1]\!]$ **do** // Iterations over the columns of A and the rows of B
  for $pos_A \in [\![A.ptr_k, A.ptr_{k+1}]\!]$ **do**
    // Iterations over the non-zero elements of the k-th column of A
    $i_A \leftarrow A.idx_{pos_A}$; // Row-index of the ($pos_A$, k) non-zero element of A
    for $pos_B \in [\![B.ptr_k, B.ptr_{k+1}]\!]$ **do**
      // Iterations over the non-zero elements of the k-th row of B
      $j_B \leftarrow B.idx_{pos_B}$;
      // Column-index of the (k, pos_B) non-zero element of B
      $C_{i_A,j_B} \leftarrow C_{i_A,j_B} + A.val_{pos_A} \times B.val_{pos_B}$;
      // Reduction (to memory) of the partial sum into the (i_A, j_B) element of C (immediate update)

// *
// * * * 'Deferred updates' version *
// Multiply phase
for $k \in [\![0, K-1]\!]$ **do** // Iterations over the columns of A and the rows of B
  $pos_{PO} \leftarrow 0$;
  for $pos_A \in [\![A.ptr_k, A.ptr_{k+1}]\!]$ **do**
    // Iterations over the non-zero elements of the k-th column of A
    $i_A \leftarrow A.idx_{pos_A}$; // Row-index of the ($pos_A$, k) non-zero element of A
    for $pos_B \in [\![B.ptr_k, B.ptr_{k+1}]\!]$ **do**
      // Iterations over the non-zero elements of the k-th row of B
      $j_B \leftarrow B.idx_{pos_B}$;
      // Column-index of the (k, pos_B) non-zero element of B
      $PO_k.row_{pos_{PO}} \leftarrow i_A$;
      $PO_k.col_{pos_{PO}} \leftarrow j_B$;
      $PO_k.val_{pos_{PO}} \leftarrow A.val_{pos_A} \times B.val_{pos_B}$;
      // The k-th partial sum of the (i_A, j_B) element of C is stored in memory in the k-th partial output
      $pos_{PO} \leftarrow pos_{PO} + 1$;
  // Each partial output is generated sorted, row-wise
// Merge phase
for $k \in [\![0, K-2]\!]$ **do** // Naive serial merge of the K partial outputs
  $PO_{k+1} \leftarrow 2DListsMerge(PO_k, PO_{k+1})$;
// PO_{K-1} is C stored in COO format
for $pos \in [\![0, length(PO_{K-1}) - 1]\!]$ **do**
  // Update C (dense) from the fully merged partial outputs
  $i \leftarrow PO_{K-1}.row_{pos}$;
  $j \leftarrow PO_{K-1}.col_{pos}$;
  $C_{i,j} \leftarrow PO_{K-1}.val_{pos}$; // Write in memory of the (i, j) element of C

Table 1.2: Comparison of Advantages and Challenges with Matrix Multiplication Algorithms

		Inner-Product (IP)	***Outer-Product*** (OP)	***Gustavson*** (Gust)
Access Count	**MV**	***A***: once ***b***: $nnz(A)$ ($\sim M\times$) ***c***: once	***A***: once ***b***: once ***c***: $nnz(A)$ ($\sim K\times$)	*N/A*
	MM	***A***: $N\times$ ***B***: $M\times$ ***C***: once	***A***: once ***B***: $M \times d(A)\times$ ***C***: $nnz(A) \times \overline{nnz_r}(B)$ ($\sim K\times$)	***A***: once ***B***: $M \times d(A)\times$ ***C***: $nnz(A) \times \overline{nnz_r}(B)$
Sequential Accesses		Accesses to A and C	Accesses to A and B	Accesses to A and C Accesses to B-rows
Input Format		Accesses to A and B benefit from alternate formats (CSR and CSC)		Allows consistent format for A and B
Parallelism Opportunity		Possible with *tiling*	Generation of *POs* over common rank of A and B	Reading A-rows and updating C-rows
Reuse Opportunity		Potentially B, but limited by its size	An A-column or B-row while computing *POs*	A set of B-rows, based on non-zero elements of A-rows
Challenges		*Intersection* search between non-zero elements in A and B	Storage for *POs* or in-place *merge* and *reduce*	*Merge* and *reduce* of *PO*-rows

CONCLUSION

There are **two main challenges** to address when computing sparse matrices. When reading irregular data from memory, addressing the ***gather*** problem is key. It mainly consists in processing efficient ***intersection* searches**. When updating irregular data from memory, addressing the ***scatter*** problem is key. It mainly consists in **managing *POs*** in memory or processing efficient **"random" *reductions***. These two challenges are tied to **IP** an **OP**, respectively, and are present in any higher rank algorithm for sparse matrix computing.

1.5.4 SpMSpM Algorithm Analysis

To better understand how the algorithms impact the number and types of memory accesses, we developed, and presented in Figure 1.6, a model based on memory accesses counts. We identify five memory access patterns: (1) those where a matrix is read only once, thus there is no opportunity for reuse, (2) where the accesses are sequential, but a *fiber* is read multiple times, (3) where *fibers* are read consecutively, but the accesses within a *fiber* are "random", (4) where the *fibers* are accessed "randomly", but within a *fiber* the accesses are sequential, and (5) where the matrix elements are accessed multiple times, and with a "random" access pattern.

For each of the three algorithms, we counted the number of read and write accesses, based on the density of the input matrices, and the local memory storage available ("OD", "1D" and "2D"), as resumed in Table 1.3. In Figure 1.6, we plotted the number of memory accesses in logarithmic scale for SpMSpM, based on a square matrix with $M = 10,000$. Indeed, the number of non-zero elements in C depends on the structure of the input matrices and in this model, we have assumed the non-zero entries in the input matrices are uniformly, randomly distributed. This corresponds to a *worst case* for sparse matrices (no spatial and temporal locality). Banded input matrices would, for example, have fewer non-zero elements in C. In each graph, we have separated the accesses into A (*bottom*), B (*middle*) and C (*top*). The color code shows the access pattern, based on the five categories described above.

In the top-row of Figure 1.6 (graphs ①, ② and ③), we start by considering a naïve system which has no local storage ("OD"), thus all data comes from main memory. From these figures, we clearly see that **IP** requires more accesses at low density, as the A-matrix must be read N times (graph ①). Whereas for **OP** and **Gust**, the A-matrix is read only once (thus the *grey* color). For **OP** and **Gust** the total number of accesses is the same, however, with **OP** (graph ②), the accesses to C are "random" (*red*). With **Gust** (graph ③), the accesses to B are sequential within the rows (*orange*) and the C-rows are generated sequentially (*yellow*), thus **Gust** avoids having any fully "random" accesses (*red*).

Since main memory accesses are the principal bottleneck, systems integrate local storage

Table 1.3: Memory Access Count for Each Matrices of `IP`, `OP` and `Gust` SpMSpM

	Inner-Product (`IP`)	***Outer-Product*** (`OP`)	***Gustavson*** (`Gust`)
$\mathbf{A}(M,K)$	**0D**: $N \times nnz(A)$ **1D-2D**: $nnz(A)$	**0D-1D-2D**: $nnz(A)$	**0D-1D-2D**: $nnz(A)$
$\mathbf{B}(K,N)$	**0D-1D**: $M \times nnz(B)$ **2D**: $nnz(B)$	**0D**: $M \times d(A) \times nnz(B)$ **1D-2D**: $nnz(B)$	**0D-1D**: $M \times d(A) \times nnz(B)$ **2D**: $nnz(B)$
$\mathbf{C}(M,N)$	**0D-1D-2D**: $nnz(C)$ (max: $M \times N$)[1]	**0D-1D**: $2 \times M \times d(A) \times nnz(B)$ **2D**: $nnz(C)$ (max: $M \times N$)[1]	**0D**: $2 \times M \times d(A) \times nnz(B)$ **1D-2D**: $nnz(C)$ (max: $M \times N$)[1]

[1] We can not easily quantify $nnz(C)$ knowing $nnz(A)$ and $nnz(B)$. We thus use a statistic approximation.

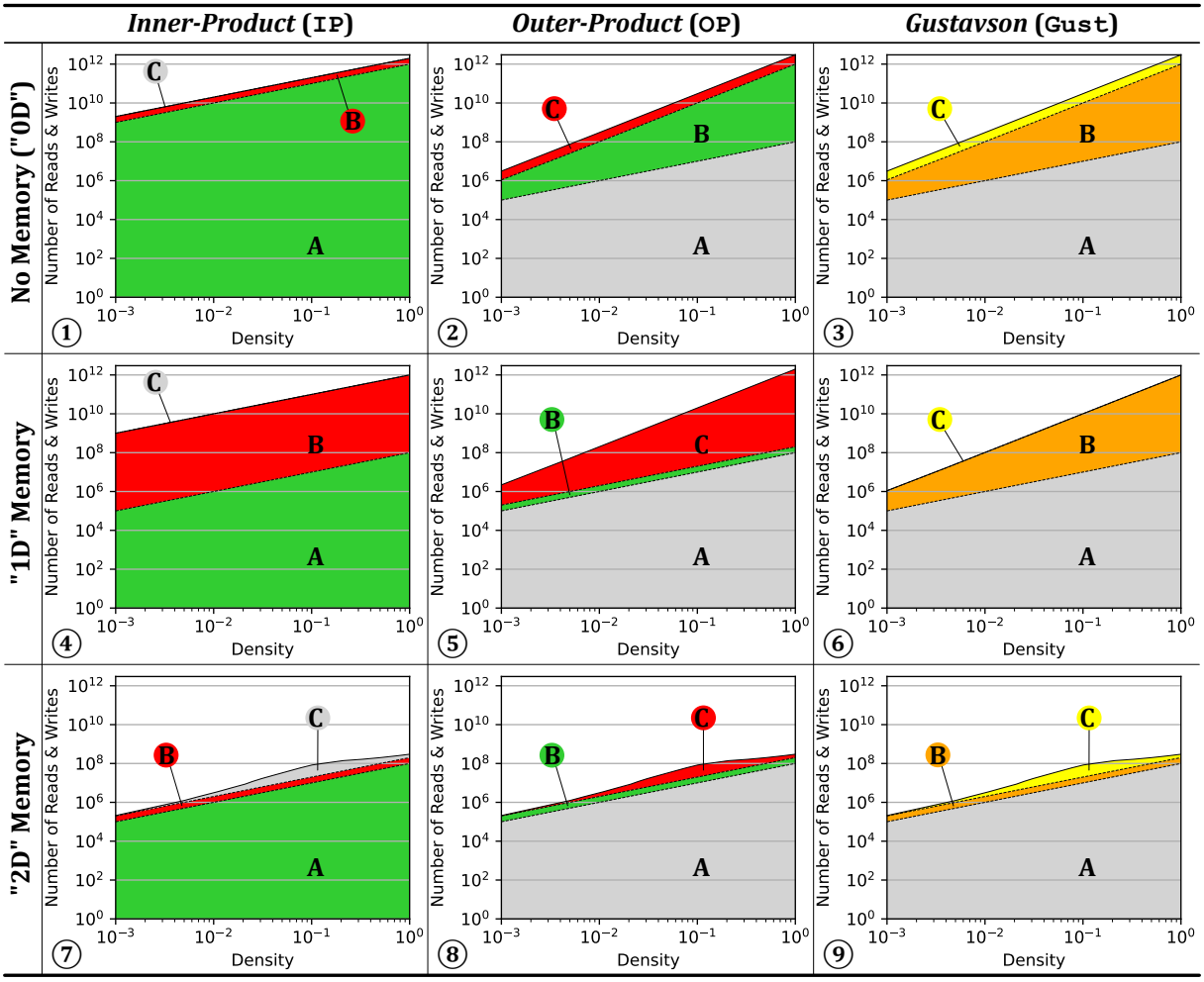


Figure 1.6: Model of Memory Access Counts Depending on Algorithm and Local Memory Size

(such as buffers or caches). Thus, we consider the case of an system with sufficient local memory to store one full row, per matrix (few sparse rows, "1D"). We assume this local storage is perfect (no misses) and we re-calculate the number of remaining main memory accesses, as shown in the middle-row of Figure 1.6 (graphs ④, ⑤ and ⑥). The matrices are assumed to be initially stored in main memory, thus they must be accessed at least once. As can be seen, for **IP** (graph ④), the number of A-accesses is reduced, because rows are reused, thus the total number of A-accesses becomes equal to that in **OP** and **Gust**. For **OP** (graph ⑤), the single row buffer greatly reduces the number of B-accesses, however, it does nothing for C (despite the appearance, due to the logarithmic scale). For **Gust** (graph ⑥), the single row buffer is sufficient to cache the *reduction* operations in C, and thus, it now has fewer overall accesses than **OP**. It is also interesting to note that **OP** and **Gust** scale better than **IP** with lower densities, as seen by the shallower slope.

Finally, we have considered the extreme case where an system has a buffer of sufficient size to store two sparse matrices ("2D"), which is shown in the bottom-row of Figure 1.6 (graphs ⑦, ⑧ and ⑨). Although this case is not practical for large matrices, through the proper use of *tiling* to locally reduce the dimensions of a sub-matrix, this case can be achieved at the level of a *tile*. In this case, the total number of external accesses is equal for the three algorithms, as each matrix is accessed only once from main memory. For **IP** (graph ⑦), it is interesting to note that such a large buffer is needed to locally address the *intersection* search in B. Similarly for **OP** (graph ⑧), a large storage is required to address the *reductions* in C. For **Gust**, in fact, it is not necessary to buffer the entire B-matrix. Indeed, buffering a few rows (corresponding to $nnz_r(A)$) is sufficient to achieve the number of accesses shown in graph ⑨. This corresponds to an intermediate design point (multiple row buffers for B) not explicitly shown in the figure (between graphs ⑥ and ⑨).

When viewed overall, Figure 1.6 highlights the benefits of **Gust**. We clearly see that **Gust** does not require any fully "random" accesses (*red*). With a single row buffer, it is possible to address the *reductions* in C. And, with a limited number of row buffers, the number of external memory accesses for B can be reduced. Based on this analysis, it is clear that, among the three base algorithms, **Gust** is the best candidate for hardware implementation of SpMSpM.

NOTE

This analysis is extremely high level: we do not consider the real behavior of memory instances such as caches; we do not consider the structure of matrices. We do more precise analysis later in this manuscript.

CONCLUSION

Based on a basic analysis of the memory accesses on SpMSpM, the ***Gustavson*'s algorithm** appears as the **best trade-off** to minimize memory traffic with reasonable amount of storage. Second best is **OP** which behaves better than **IP** at low density.

1.6 Conclusion

In this chapter, we established the foundation for understanding the relationship between applications working with irregular data and the underlying challenges in hardware. Irregular accesses are common in sparse computing and cause reduced cache efficiency, limited parallelism, and higher memory latency exposure. We analysed the most representative algorithms for sparse matrix multiplication kernels, and identified their key operations that suffer from irregularities. *Intersections* and *reductions* on irregular data with classical systems are costly, thus being the central challenges of sparse computation, and driving motivation for designing specialized hardware as we show in the next chapter.

TAKEAWAYS

Sparse matrices: large, highly sparse, structured (such as banded matrices)
Sparse formats: diverse, reduce memory footprint
- COO, the simplest
- CSR and CSC, the mostly used
- BaCSC, our own sparse format for banded matrices

Sparse kernels: sparse MV and MM multiplications (such as SpMV and SpMSpM), generate indirect accesses
Algorithms: *Inner-Product* (`IP`), *Outer-Product* (`OP`) and *Gustavson* (`Gust`), extended with *tiling* methods
- Best potential to reduce memory transfers: `Gust`, followed by `OP` at low density

Problem: irregular accesses, generate high memory transaction latencies
Challenges:
- *gather*: *intersection* searches
- *scatter*: "random" *reductions*, or storing and *merging partial outputs* (*POs*)

Chapter 2

State-of-the-Art

Contents

2.1 Introduction

TODO

From SoA paper

2.2 Hardware Accelerators for Sparse Matrices and Vectors

In this section, we describe the recent (2018-2023) hardware accelerators for sparse MM and MV, targeting ASIC-type implementation. We try to be consistent on notations and architecture schemes to simplify the comparison. Before going into the detailed overviews of the accelerators, in Table 2.1 we present a criteria-based classification of the designs.

A first criterion to classify the accelerators is between *standalone* accelerators that address the full kernel without the assist of a processor (*e.g.* OuterSPACE [70] – Section 2.2.1) and ones that address only a part of the kernel and need a processor to manage the computation (*e.g.* PHI [66] – Section 2.2.9). *Standalone* accelerators are directly connected to the main memory and internally manage their own custom memories. The other accelerators actually intervene at different levels of the memory hierarchy, giving a second classification criterion. Certain accelerators intervene near the processors (*e.g.* ASA [100] – Section 2.2.8), near the cache (*e.g.* PHI and SortCache [84] – Sections 2.2.9 and 2.2.7), or near the main memory (*e.g.* SPiDRE [10] and SpaceA [97] – Section 2.2.9).

Each accelerator is optimized for a specific algorithm (**`IP`**, **`OP`** or **`Gust`**), our third criterion. For example, some accelerators (*e.g.* PHI and SortCache) address the *reduction* problem and are adapted to **`OP`** algorithms but *reductions* are also required with **`Gust`** and highly-*tiled* algorithms. ExTensor [40] (Section 2.2.2) is a general purpose accelerator that is demonstrated for a highly-*tiled* **`OP`** but also makes major contributions for **`IP`**.

Table 2.1: Algorithmic and Architectural Classification of Existing Accelerators

Name	Date	Autonomy: *Standalone*	Autonomy: *Processor Assist*	Position in Memory Hierarchy: *Near Processors*	Position in Memory Hierarchy: *Near Cache*	Position in Memory Hierarchy: *Near Main Memory*	Most Adapted Algorithm: *Inner-product*	Most Adapted Algorithm: *Outer-product*	Most Adapted Algorithm: *Gustavson*	Hardware Support for: *Gather*	Hardware Support for: *Scatter*	Sparse Kernels Addressed
OuterSPACE [70]	2018	■		N/A				■		■	■	MV, MM
ExTensor [40]	2019	■		N/A			■	[a]		■	[a]	MV[b], MM
PHI [66]	2019		■		■			■	[b]		■	MV, MM[b]
SPiDRE [10]	2019		■			■	■			■		MV, MM[b]
MatRaptor [86]	2020	■		N/A					■	■	■	MM
SpArch [104]	2020	■		N/A				■		■	■	MM
SpaceA [97]	2021	■				■	■			■	■	MV
Gamma [101]	2021	■		N/A					■	■	[a]	MM
InnerSP [6]	2021	■		N/A					■	■	■	MM
SortCache [84]	2021		■		■			■	[b]		■	MV[b], MM
ASA [100]	2022		■	■				[b]	■		■	MV[b], MM
Flexagon [65]	2023	■		N/A			■	■	■	■	■	MM
GSCSp [57]	2023	■		N/A					■	■	■	MM

Gray cell: the accelerator meets the criterion.
[a] Not main contribution but addressed in the article.
[b] Could be adapted but not discussed in the article.

From a memory perspective, the two main problems are *gather* and *scatter*, our fourth criterion. Certain accelerators offer solutions for one, the other or both. For example, some accelerators (*e.g.* SpArch [104] – Section 2.2.3) are specialized for an **OP** algorithm but optimize both *gather* and *scatter*. Finally, accelerators preferably focus on sparse MM, MV or both – our final criterion.

After this first comparison, in Sections 2.2.1 to 2.2.6 we present the *standalone* accelerators, followed by the *processor assist* accelerators in Sections 2.2.7 to 2.2.8. In Section 2.2.9, we briefly touch on other accelerators.

2.2.1 OuterSPACE

OuterSPACE [70] is an **OP** accelerator with a focus on SpMSpM kernels, although it can also handle SpMV. It is a highly parallelized architecture which employs Single-Program Multiple-Data (SPMD)-style *processing elements* (*PEs*) that fetch data from main memory through a two-level highly-banked cache hierarchy as shown in Figure 2.1. The input A- and B-matrices are stored in *(UOP, CP) column-* and *row-major* formats, respectively, to match the read access patterns of the **OP** algorithm. The *partial* and final outputs are stored in *(UOP, CP)* format.

The computation is temporally divided into the *multiply* and *merge* phases (Figure 2.2). During the first phase, inputs are divided into *tiles* of several rows (A) and columns (B) and distributed through the *processing tiles* (*PTs*) by a *control unit*. Within each *PT*, each *PE* processes multiplications of one element of the k^{th} A-column with all elements of the k^{th} B-row and an L0 cache in the *PT* facilitates the reuse of the B-rows between *PEs*. This phase generates a *partial output* (*PO*) per *PT*, corresponding to the *tile* of rows (A) and columns (B) that it was assigned.

Then, during the *merge* phase, the *POs* are combined to generate the final C-matrix. Each *PT* locally sorts the *POs*, and then the *PTs* work together to merge them. This algorithm was chosen to maximize data locality and parallelism, although it is less efficient. This *merge* phase only uses half of the *PEs* so the others are disabled (clock-gated) and the caches are reconfigured in a scratchpad mode to save energy.

Note, if NNZ elements in the *POs* exceeds the L0 cache capacity, it overflows to the last level cache, which results in reduced performance. OuterSPACE performs best when the $NNZ_c(A)$ and $NNZ_r(B)$ are both uniform which ensures a uniform distribution of work across the large number of *PTs* and *PEs*.

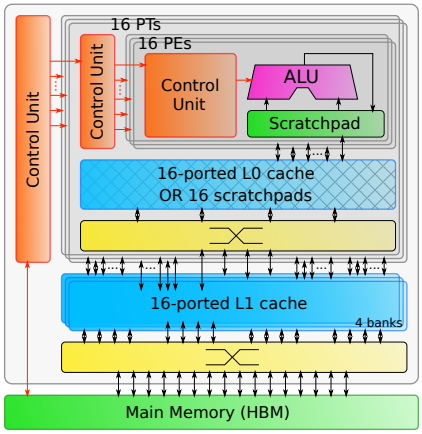


Figure 2.1: Architecture of OuterSPACE

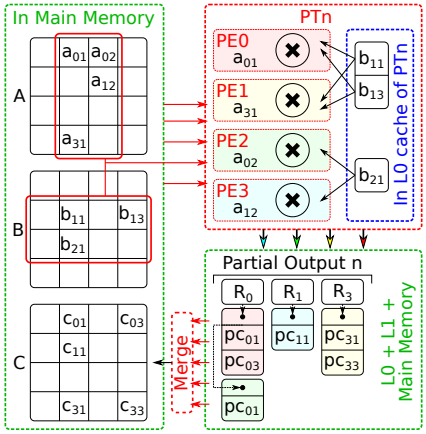


Figure 2.2: OuterSPACE Data Flow

2.2.2 ExTensor

The authors created ExTensor [40] as a general purpose sparse tensor algebra accelerator (including SpMSpM). It is an **OP** accelerator but due to its support for three levels of fixed *tiling* of the inputs, it optimizes the *intersection* search between non-zero elements or *tiles*[1]. By doing the *intersections* ahead of time, at different levels of the memory hierarchy (which correspond to the *tile* levels), ExTensor is able to efficiently eliminate *ineffectual operations*, thus addressing the main issue in **IP** or complex *tiling* algorithms. Indeed, the input matrices would typically be stored in an *(UOP, UOP, UOP, CP)* format, corresponding to three levels of *tiling*. ExTensor is designed with a flexible architecture composed of different bricks (see Figure 2.3). To simply aid in the understanding, the interactions between the bricks are described using function calls:

- The *Scanner* is a storage unit that delivers a stream of coordinates in increasing order, fetching them from memory. The coordinates are delivered through the *Iterate()* operation call.
- The *Intersect* is a hardware block that co-iterates over several *Scanners* in parallel and compares the coordinate streams. This unit performs the *intersection* search required by **IP** type algorithms, delivering a single stream with the matching coordinates, which are the coordinates of non-zero *partial sums*. To optimise the *intersection* step, it is possible to skip a range of coordinates in a stream depending on the current coordinate of another stream. Thus, the *Intersect* sends skip messages with *SkipTo()* operations to the other *Scanners*.
- The *Coordinator* is a hardware unit that includes *Scanners* and *Intersect* units, and manages the input and output data depending on the current processed *tile*, as shown in Figure 2.3. Data and *metadata* are respectively stored in storage units and *Scanners*, while a *Sequencer* manages the *Iterate()* calls so that the *Intersect* brick delivers the stream of effective coordinates. In addition, in the *Tile Coord* brick, the offsets of the *tile* are added back, so the output of the *Coordinator* is in absolute coordinates.

These bricks can be placed at different positions in a memory hierarchy, depending on the *tiling* and selected hardware. Thus, the *Coordinator* is able to intersect at coarse-granularity with output data being *metadata* of the next *tile*, skipping an entire *tile* when appropriate, with low computation cost, and also at fine-granularity when output data are the non-zero elements that need to be multiplied.

ExTensor is designed with three levels of *tiling* that each have either input or output sequentiality. First, a *Sequencer* plus *Scanners* block ① is placed close to the main memory to determine which *tiles* to send to the next storage, the *Last Level Buffer* (*LLB*). Then, for each *tile*, a *Coordinator* ② in the *LLB* intersects the next level *tiles*, which are sent to several *PEs* that will process in parallel. Finally, in each *PE*, a *Coordinator* ③ intersects the non-zero elements of the computed *tile* and sends them to the arithmetic unit of the *PE* for multiplication.

In addition to these bricks which are the main contributions, ExTensor still needs to perform the *merge* phase, including the *reduction* of the *POs* which are stored in *CP*[2] format. The *Partial Output Buffer* (*POB*), is managed by *tiles*, and the content of the *tile* is stored on-chip using a Content Addressable Memory (CAM) for quick access, enabling *reductions* to

[1]Tensauraus [87] is another accelerator designed for tensors, but their focus is on SpMM.

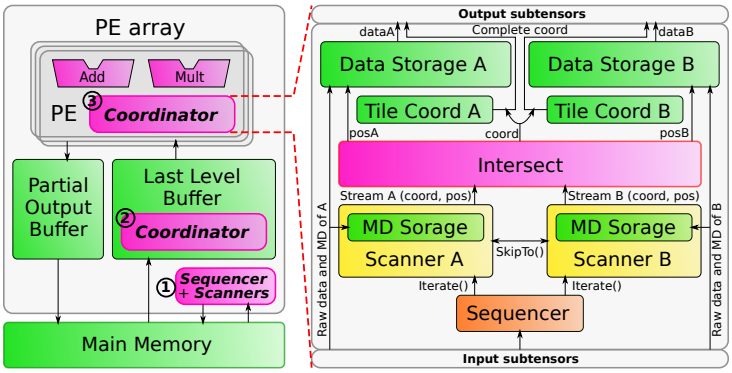


Figure 2.3: Architecture of ExTensor

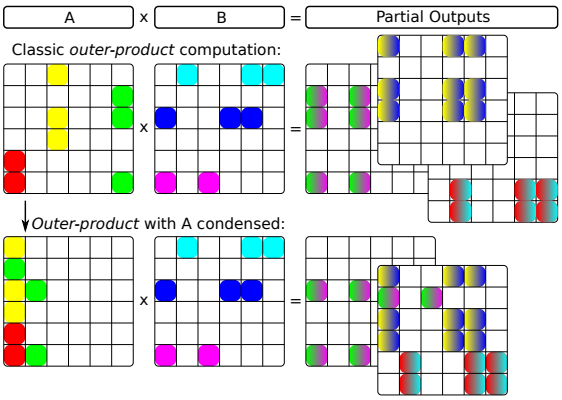


Figure 2.4: *Outer-product* with a Condense A-Matrix

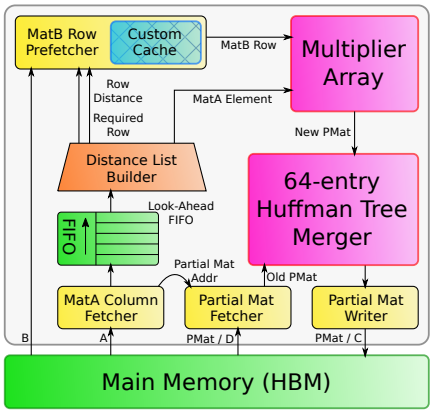


Figure 2.5: Architecture of SpArch

be performed. On overflow, the *tiles* are flushed to main memory. To benefit from ExTensor's support for *tiling*, the input matrices need preprocessing, making it difficult to directly compare performance with other accelerators.

2.2.3 SpArch

SpArch [104] uses an **OP** algorithm for SpGEMM in order to benefit from the simplified access pattern for the inputs. To address the management of the large volume of *POs*, SpArch pipelines the *multiply* and *merge* phases and uses condensing methods to reduce the number of *POs* and scheduling methods to reduce memory traffic. Unlike other **OP** accelerators [70], with SpArch, both A and B are stored in *(UOP, CP) row-major* format. As a consequence of this *CSR* format, the A-rows can easily be condensed to the left (see Figure 2.4). Thus, when a condensed A-column is read, it will require reading all the B-rows corresponding to the different A-columns that have been combined. The *MatA Column Fetcher* reads the combined A-columns and the *MatB Row Prefetcher* reads the necessary B-rows (see Figure 2.5), thus masking the memory latency. The *MatB Row Prefetcher*, stores multiple B-rows in a cache, and it uses information about the upcoming A-columns (provided by the *Distance List Builder*) to ensure the needed B-rows are kept in the cache.

A key contribution of SpArch is the merge unit that is able to merge sixty-four *POs* in parallel, through a binary tree type architecture. The core of the merger is a brick which compares and merges $4{\times}4$ $(index, value)$ pairs in one cycle. These are combined to build a hierarchical merger array whose output is then written to main memory, and read back for further merging by the *Partial Matrix Fetcher*. The authors note that the order in which the *reductions* are performed is important, specifically sparser *POs* should be merged first and they propose a Huffman tree scheduler to determine the best order to minimize memory accesses. Overall, SpArch is able to compute an **OP** SpGEMM[2] kernel with reduced main memory traffic for the *POs* mainly by addressing the *scatter* problem with a highly parallelized merger.

[2]Since the merger can read *POs* from memory, the D-matrix can be treated like an initial *PO*.

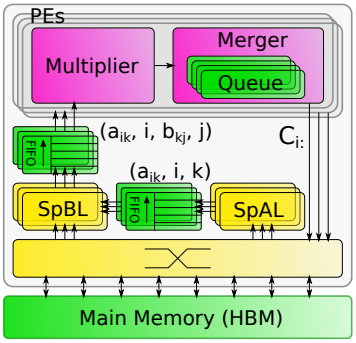


Figure 2.6: Architecture of MatRaptor

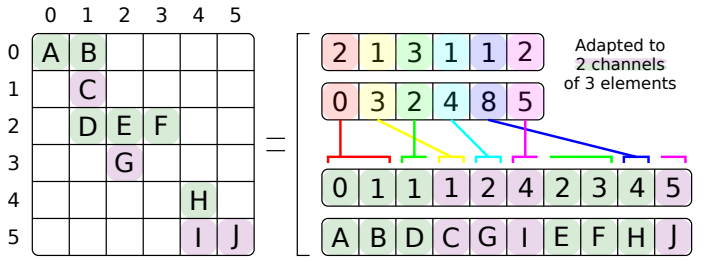


Figure 2.7: *(FL, UOP, CP) Channel-Major* Format (C^2SR)

2.2.4 MatRaptor

MatRaptor [86] is an accelerator designed to compute SpMSpM kernels. It is based on **Gust** algorithm[3], which makes it possible to process multiple A-rows (2 to 8 for MatRaptor) in parallel. In MatRaptor (see Figure 2.6) a *SpAL* (*Sparse Matrix A Loader*) fetches non-zero elements of a A-row and sends them to a *SpBL* (*Sparse Matrix B Loader*). With the *metadata* of this element, the *SpBL* fetches non-zero elements of the corresponding B-rows and sends pairs of A- and B-elements to a *PE*. This *PE* computes the *partial sums* and sends the results into queues for the *merge* process. Instead of separating *multiply* and *merge* phases, several queues are used to store *partial sums* and partially merged *partial sums*, allowing the two phases to be pipelined. The *PE* merges all the *partial sums* resulting from an entire A-row, before sending the final output (the corresponding C-row) to main memory.

In MatRaptor, multiple A-rows are processed independently and in parallel in different *PEs* and each *PE* is assigned to a channel in an HBM memory [47]. It is important that each *PE* can read its A-rows without interfering with the other *PEs*. With the classic *(UOP, CP) row-major* format, the rows are consecutive in memory and are not aligned to channel boundaries. To solve this problem, the authors propose a new format called C^2SR (*(FL, UOP, CP) channel-major*) that is based on a *(UOP, CP) row-major*. For example, with two *PEs*, all the elements of the even rows are stored in one channel, and the elements of the odd rows, in another channel, thus isolating the data. With this approach, the row pointers (*UOP*) are no longer monotonic, which means that the length of a row can not be deduced by subtracting adjacent row pointers. This requires the addition of a new table (which we call *fiber length FL*), which stores the length of the non-zero elements in each row (see Figure 2.7). This modified storage format enables multiple *PEs* to make effective use of the HBM.

MatRaptor is one of the first accelerators to use **Gust** algorithm, however, it has some limitations regarding effective reuse of B-rows, both between parallel *SpBLs* and temporally, as A is traversed. A key contribution is the proposed C^2SR input format, which makes it possible to separate rows into memory channels, to improve performance.

2.2.5 InnerSP

InnerSP [6] is a SpMSpM accelerator based on the **Gust** algorithm[4]. The authors of InnerSP justify their choice by showing quantitatively that with the **OP** the *POs* represent a large volume of data, and that for many matrices the *reductions* can not be done fully on-chip and thus must be buffered in main memory. Normally, with **Gust** algorithm, B must be read repeatedly, but InnerSP proposes a special cache for B which exploits the spatial locality.

The architecture of InnerSP is shown in Figure 2.8. The *A reader* reads the A-rows. Two separate caches are used for B, one that stores the row pointers and the other which stores column indices and values. The *B reader* reads the required B-rows, hopefully getting the data from the caches. Parallel multipliers perform the multiplications and then a hash-unit generates a hash based on the indices in C. A *reduction* unit based on hash tables contains parallel comparators which search for matching hash values and performs the *reductions*. When all the operations for a A-row are completed, the *C writer* writes the C-row back to main memory.

[3] Called *row-wise product* in [86].
[4] The authors call this *row-wise inner-product*, or confusingly sometimes *inner-product*.

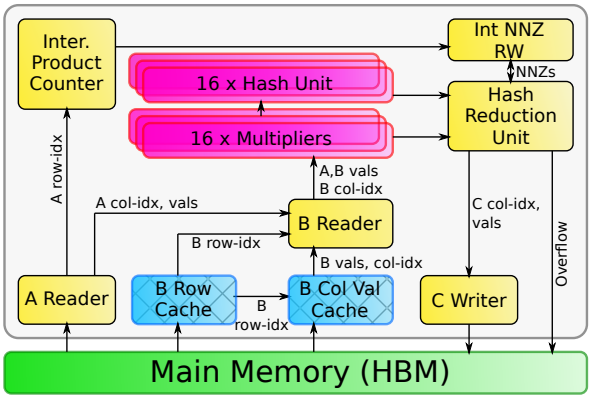


Figure 2.8: Architecture of InnerSP

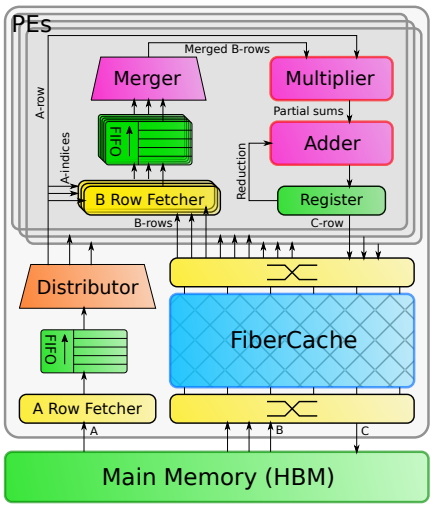


Figure 2.9: Architecture of Gamma

The architecture is efficient, as long as the *Hash Reduction Unit* does not overflow. When overflows occur, the extra entries are temporarily stored in main memory, but there is a significant performance hit. To minimize the chance of overflows, InnerSP uses a pre-scanner to estimate the number of non-zero entries in the upcoming C-rows. The estimate primarily considers the NNZ in the A-rows. If the estimated number of non-zero entries in C exceeds the capacity of the *Hash Reduction Unit*, then the row is split and processed in two passes. Conversely, if the number of non-zero entries in C is small, two A-rows can be processed simultaneously.

To improve the performance of the B-cache, the authors exploit a technique from [8]. The non-zero elements in the A-rows provide direct information about which B-rows are required. The column index table of A is already pre-fetched by the *A reader*, and when a row must be evicted from the B-cache, it chooses the ones which will not be required based on the upcoming entries in the column index table of A. This technique (also used by SpArch [104]) maximizes the reuse of B-rows.

InnerSP is an accelerator based on **`Gust`** algorithm which achieves high reuse of the B-rows, through a look-ahead in the A-*metadata*.

2.2.6 Gamma

Gamma [101] is a dedicated accelerator for SpMSpM kernels, which uses **`Gust`** algorithm. The two input matrices are both stored in *(UOP, CP) row-major* format and the output is generated in the same format, providing consistency.

The Gamma architecture, presented in Figure 2.9, is composed of a fetcher for the A-rows that contains a *Distributor* to distribute them to 32 *PEs*. Each *PE* processes an entire A-row and computes the corresponding C-row. The *PE* fetches the required B-rows and merges and sorts them in a 64-wide *merge* block, before multiplying with the A-elements. As a result, the output C-row is generated in order. The *reduction* step is, thus, highly simplified, because any duplicate indices are adjacent. Normally, the C-row can be written back to main memory. However, if the A-row contains more than 64 non-zero elements, then it must be processed with multiple passes. After each pass, the partial C-row is stored temporarily. Finally, the temporary C-rows are read-back and merged by the *PEs*. Of course, the B-rows must be accessed repeatedly by the different *PEs*, and Gamma proposes *FiberCache* which is a highly banked cache which fetches the B-rows in advance, based on the column index table of A. It keeps track of which rows of B are most needed by the *PEs*, and when an eviction is necessary, the victim, is the row which is least needed.

Gamma encounters some difficulties when the A-matrix has rows that are too dense, requiring multiple iterations through the *PEs*, which creates a high-load on the cache. The authors propose a software preprocessing technique which splits dense A-rows and rearranges them to make best use of the *FiberCache*.

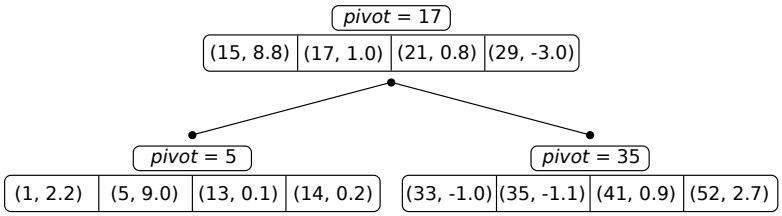


Figure 2.10: VBST Data Structure Used by SortCache

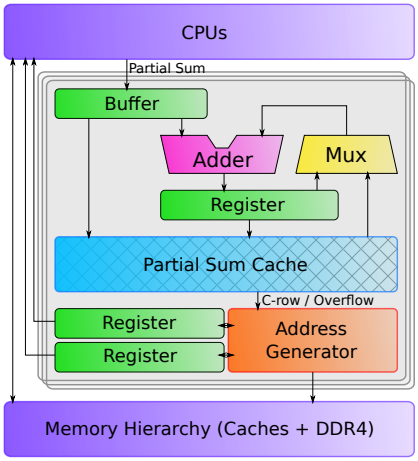


Figure 2.11: Architecture of ASA

2.2.7 SortCache

The authors of SortCache [84] note that modern processors are able to load an entire cache line in parallel but in many sparse applications less than 50% of the data within a line is used, even with hand-tuned code. They propose a light SpGEMM accelerator which offloads the *reduction* step, based on the *Vectorized Binary Search Tree* (VBST), a data-structure from their earlier work [83, 85]. The VBST is a standard binary search tree where the size of node is a multiple of the cache block size and contains a sorted vector of K $(key, value)$ pairs. Associated with each node is a *pivot*; the pivots, with left and right child pointers, form a binary tree, as illustrated in Figure 2.10.

Using the VBST, SortCache focuses on accelerating the *scatter* updates to the output matrix, which are performed directly in the cache. The authors augment the processor with a single new instruction, *RED Key, Value*. After K updates have been issued, the hardware launches a *reduction* in the SortCache, where the K $(key, value)$ pairs are inserted into the VBST. This is done by traversing the tree from the root to the leaves. When duplicate keys are found, the *reduction* operation is performed in the cache. When new keys are found, they are inserted, which can result in a temporary, new node with up to $2K$ $(key, value)$ pairs. This temporary node is partitioned, with one half having exactly K entries. The insertion procedure may require visiting each level of the tree at least once ($\mathcal{O}(log(n))$ and it does not ensure the uniqueness of the keys, so at the end, an additional full traversal of the tree is required to remove duplicates.

The nodes closest to the root of the tree are accessed most frequently and are thus mapped to the caches closest to the processor. The authors propose to re-purpose cache tracking hardware (e.g. TAG values, LRU counters) to store the additional metadata (pivots, pointers). One of the difficulties with SortCache is that, in some cases, a large fraction of the VBST nodes are underutilized.

2.2.8 ASA

In the scope of graph analytic problems, the authors of ASA [100] observed that **`Gust`** algorithm achieves high performance for SpGEMM kernels but note that it still requires acceleration for the *reduction* step[5]. The software state-of-the-art for *reduction* uses hash tables [16]. Some hardware accelerators, such as HTA [103], enhance performance for hash operations, but those solutions are not optimized for SpGEMM kernels. ASA is a low area, near-core extension for a general purpose processor that specifically accelerates the *reduction* step for **`Gust`** algorithm.

ASA accelerates *reduction* operations received from software via custom instructions. As presented in Figure 2.11, the *partial sums* from these instructions are stored in a waiting buffer as $(index, value)$ pairs with a key that is a hash of the index, computed in software to maintain flexibility. The cache line, accessed with the key, is filled with the indices and the

[5]The authors refer to the *reduction* step as SPA (Sparse Accumulation).

partial sums. If a corresponding *partial sum* is already stored, ASA accumulates it and writes back the result. If not, the *partial sum* is simply stored into the cache.

To minimize die area, the cache is not dimensioned for an entire C-row[6] but ASA handles cache overflows in hardware. When a *partial sum* needs to be stored in a fully filled cache line, a victim is selected and evicted, based on a LRU policy. The evicted entry is written into a pre-allocated FIFO queue in the memory hierarchy. ASA uses an address generator to manage evicted data and updates head and tail registers for the list. Software then traverses the list to merge duplicate keys to build the final output C-row. To avoid overflows, the authors advocate *tiling* methods to efficiently distribute work through one or several ASA cores.

ASA is an accelerator which optimizes the *scatter* aspect of SpGEMM kernel with low area overhead, while leaving high software flexibility. Unlike other **Gust** accelerators, there is no hardware *gather* support for reusing or caching the B-rows.

2.2.9 Other Accelerators

In the above sections, we have described a selection of the main sparse accelerators that have appeared in the literature, covering different matrix multiplication algorithms and architectures. This list is not exhaustive, and in the following paragraphs we present several other accelerators which we will cover in less detail, due to space constraints.

2.2.9.1 SPiDRE

The authors of SPiDRE (Sparse Data Rearrange Engine) [10] propose a solution for the poor utilization of the memory hierarchy in sparse applications. They propose a near-memory data rearrangement engine. This engine can preprocess data, near memory, to create new data-structures that are dense and benefit from the memory hierarchy. For example, with a SpMV kernel, the input vector could be rearranged through a user defined function to a dense format where all non-zero elements are in the right order of computation.

2.2.9.2 PHI

The authors of PHI [66] note that many large graph algorithms favour pull (each node reads inputs from its neighbours) compared to push implementations (each node propagates results to its neighbours), as push implementations require read-modify-writes which require accessing complete cache lines, even though a small unit of data is modified. The authors include **OP** SpMV (a push algorithm) in their evaluation.

PHI is a modified cache optimized to efficiently process commutative *scatter* updates. When an update is received but the line is not in the cache, the line is not fetched, but rather marked as being in *update mode*, where it simply stores the update. If subsequent updates hit the line, they are stored, or combined via the commutative operator. When a cache line is evicted, if it contains multiple updates, PHI assumes that the coalescing has been effective, and it processes the operations via a read-modify-write from main memory. If a cache contains few updates when evicted, PHI batches these updates as $(address, value)$ tuples, allocating a new cache line to assemble the batch. Groups of updates are organized by *bins* corresponding to a small memory region. The batches are streamed to main memory. Later, the reading of the batches is also efficient, as they are contiguous.

In multi-core systems, private caches act as local buffers, thus coalescing the updates reduces coherency protocol traffic. This combination of in-cache coalescing and update buffering enables PHI to take advantage of locality when possible (like Coup [102]), but with the addition of *update batching*, PHI improves memory utilization, even when the data being updated is too large to fit in the cache.

2.2.9.3 GSCSp

GSCSp [57] is a FPGA-based accelerator based on **Gust** algorithm. In other **Gust** accelerators, a *PE* processes a full A-row but the authors note that when there is variability in the number of elements in the A-rows, *PEs* may be blocked, waiting for another *PE* to complete. Thus the authors propose to distribute the elements in an A-row to different *PEs* (*element-wise* parallelism). Initially, each *PE* buffers its output C-row, and when a *PE* advances to the next row, it pushes its partial C-row to a final merger. The authors also propose to use

[6] The authors present a *column-wise* **Gust** implementation. This choice is arbitrary and, for consistency, we stick with a *row-wise* presentation.

separate caches for the *metadata* for the B-rows and the values, and report that combined with the *element-wise* parallelism, the approach is effective for reducing the memory traffic for accessing B-rows.

2.2.9.4 Flexagon

Flexagon [65] is a SpMSpM accelerator for DNN applications. The authors claim that the optimal algorithm (**IP**, **OP** and **Gust**) depends on the matrix structure, potentially varying between DNN models and layers within a DNN, although in the majority of the cases, their data shows that **Gust** is the most efficient. Flexagon is configurable: it can implement all three algorithms. To achieve that, the authors designed a *Merger-Reduction Network* (*MRN*) that is able to perform multiplications (for *partial sum* generation), accumulations (for *reductions* operations) and comparisons (for *intersection* searches and *merge* phases) via a binary tree structure. Flexagon addresses the memory access issues of each algorithms via three custom memories: a FIFO for the A-matrix that is always read sequentially, a cache for the B-matrix that is subject to high temporal and spatial reuse in each of the three algorithms, and a PSRAM to store *POs* for **OP** and **Gust** algorithms.

2.2.9.5 Other Accelerators

In GraphR [82], the authors propose to use ReRAM technology and a cross-bar to perform graph operations directly in hardware, but this approach is not currently scalable to large problems. The ITS accelerator [77] focuses on an **OP** implementation of SpMV and they apply data-compression to the *metadata*. The authors of SPU [22] propose a more general hardware solution to input data dependencies and focus on graph applications. The focus of Alrescha [2] is an innovative and adaptive preprocessing of the matrix in hardware to simplify the accesses to the b-vector. The authors of CoSparse [29] focus on a software layer that selects the hardware acceleration that is best adapted to the given problem.

The SSSR accelerator [80] for RISC-V processors goes further by addressing *intersection* and *union* while streaming data to and from memory and it can be used for multiple kernels, however the RISC-V performance can not easily be compared with the other accelerators presented in this paper. In passing we note three other accelerators EIE [38], SCNN [71] and CANDLES [36] which have hardware support for sparsity but which are highly focused on CNNs, besides SDMA [32] that focuses on GNNs.

The SpaceA [97] accelerator exploits near-memory computing inside a 3D DRAM memory to implement SpMV using an **IP** algorithm. With preprocessing, the rows of the matrix are assigned to banks within specific DRAM dies, while optimizing spatial locality. A separate die is used for storing the input and output vectors. Associated with each DRAM bank which stores the matrix, there is a *PE* which reads the non-zero values of the matrix, fetches the required vector entries and performs the computations. Bringing the compute near the DRAM and exploiting emerging 3D technology, is an orthogonal approach to reduce memory access overheads.

2.2.9.6 Other FPGA based Accelerators

Many sparse accelerators have been prototyped on FPGAs [64]. Indeed, AMD/Xilinx provide a turnkey library for SpMV [21, 58]. In [26], the authors propose a higher-performance SpMV using a modified *CSR* format where the data is packed for efficient, streaming access in HBM[7]. They also propose a highly-banked on-chip memory for the vector updates, and a hazard resolution strategy that allows efficient pipelining.

2.3 Comparison of Existing Accelerators

In this section, we analyze and further compare the accelerators that were described in Section 2.2. We start with Table 2.2 where we summarize the accelerators, chronologically, showing the research group where they were developed. We see that MIT is very active in the field, as they were involved in developing four different accelerators (*e.g.* ExTensor, PHI, SpArch and Gamma).

[7] GraphLily [41] is a precursor of HiSparse, also from Cornell University.

Table 2.2: Chronological Summary of Sparse Accelerators

Name	Date	Institute / University	Summary
OuterSPACE [70]	2018	Univ. of Michigan / Arizona State Univ.	Highly parallelized SPMD, two levels cache – **OP** SpMSpM
ExTensor [40]	2019	Univ. of Illinois / NVIDIA / MIT	General purpose tensor algebra – With *tiled* SpMSpM
PHI [66]	2019	MIT (CSAIL) / Carnegie Mellon Univ.	In cache, hash-based accelerator for *reductions*
SPiDRE [10]	2019	UPC, BSC (Spain) / Arm Research	Near main memory data reorganisation for *intersections*
MatRaptor [86]	2020	Cornell	Row-parallel with HBM-aware custom format – **Gust** SpMSpM
SpArch [104]	2020	MIT / Stanford / NVIDIA	Compression method to reduce *POs*, *multiply/merge* pipe – **OP** SpGEMM
SpaceA [97]	2021	UC Santa Barbara	In memory accelerator for 3D DRAM – **IP** SpMV
Gamma [101]	2021	MIT (CSAIL)	Parallelized architecture with *FiberCache* for B – **Gust** SpMSpM
InnerSP [6]	2021	KAIST / DGIST (S. Korea)	A-look-ahead, caches for B, *Hash Reduction Unit* for C – **Gust** SpMSpM
SortCache [84]	2021	Georgia Tech / Zettaflops / Sandia Labs	In cache accelerator for *reductions*, based on the VBST
ASA [100]	2022	Lehigh Univ. / Lawrence Berkeley	Custom cache accelerator for *reductions* – **Gust** SpGEMM
Flexagon [65]	2023	Univ. of Murcia / Georgia Tech / NVIDIA	Configurable accelerator supporting all three algorithms – SpMSpM
GSCSp [57]	2023	Nanyang Technological Univ.	Element-wise, FPGA-based accelerator – **Gust** SpMSpM

2.3.1 Benchmarks, Evaluation and Performance

Up to this point, we have discussed the architecture of the various accelerators, without reporting the performance they achieve. Most of the matrices used for benchmarking come from the SuiteSparse Matrix Collection [52] that contains a large number of matrices from diverse, real applications. Domain specific matrix libraries are also used [5, 54, 74, 81], as well as some synthetic matrix generators [1, 12]. The densities are in general less than 1% and can be as low as 10^{-5}%. ExTensor, SortCache and Flexagon also benchmark with denser matrices. In all cases, large matrices with several million NNZ, exceeding the size of a normal cache, have been evaluated.

In Table 2.3, we note any required preprocessing, assuming, arbitrarily, that matrices are initially stored in *CSC* format[8]. We see that a few accelerators require a conversion to their specific format (*e.g.* ExTensor and MatRaptor). PHI, SortCache and ASA are not concerned by preprocessing since they only address *scatter*. In any case, fair benchmarking must take into account the cost of specialised preprocessing.

Not all accelerators have reached the same level of design maturity, and in Table 2.3, we have shown how each design was analyzed or simulated. All designs report that they have been simulated at a cycle-accurate level. Most of the accelerators use custom simulators or tools like *gem5* [60], *ZSim* [78] and STONNE [67], but unfortunately there is no common approach, making it difficult to fairly compare the reported performance. RTL descriptions are often only generated for key parts of the design. To estimate power and area, authors used tools such as CACTI [9] and McPAT [56][9]. In the last column of the table, we show the size of the local memory storage used in simulation by each accelerator. Except for SPiDRE and SpaceA which work directly in main memory, the local memories are in the range of 0.3MiB to 38MiB, corresponding to 27k to 2.8M entries. Thus, in the model presented in Section 1.5.4, most of the accelerators are situated in the middle-row of Figure 1.6, with a local memory corresponding to a few matrix rows.

The accelerators performance are reported as a speedup versus a baseline, which may differ according to their hardware and software platforms: typically the Intel MKL library [43] and the Armadillo library [79] for CPUs, or cuSPARSE [69] and CUSP [23] for GPUs. Since different baselines with different numbers of threads and cores are used, these speedups need to be interpreted with care. The speedups are in general higher for the *standalone* accelerators as they provide optimized hardware for the entire kernel, while *processor assist* accelerators have less impressive speedups but aim to be general purpose and have lower area overhead.

[8]This choice is based on the SuiteSparse Matrix Collection where matrices are stored *column-major*.

[9]We have not attempted to compare power/energy as the data in the literature is heterogeneous and can not be fairly compared.

Table 2.3: Qualitative Analysis and Evaluation of Sparse Accelerators

Name	Preprocessing[1]	Functional Evaluation (Tool / Evaluation Specifics / Host[2])	Local Memory Size
OuterSPACE [70]	$A \rightarrow$ *row-major*	*gem5* / Model only key elements; Skip memory allocation; Ignore start-up time / ∅	528KiB (33.8k entries)
ExTensor [40]	A, $B \rightarrow$ highly *tiled* custom format	Custom Python simulator / Only the *Coordinator* is evaluated; Remainder modeled analytically / ∅	∼38MiB (2.5M entries)
PHI [66]	*N/A*	*ZSim* / Entirely evaluated / 16-cores system with 3 level cache	32.3MiB (2.8M entries)
SPiDRE [10]	Not needed	*gem5* / *Unknown* / Single ARM core	∅ (main memory size)
MatRaptor [86]	A, $B \rightarrow C^2SR$	*gem5* / Entirely evaluated / ∅	320KiB (27.3k entries)
SpArch [104]	A, $B \rightarrow$ *row-major*	Custom C++ simulator / Entirely evaluated / ∅	940KiB (62.5k entries)
SpaceA [97]	$A \rightarrow$ row-aligned to DRAM banks	Custom simulator / Simulation tracking values stored in 3D-DRAM / ∅	4GiB (268.4M entries)
Gamma [101]	A, $B \rightarrow$ *row-major*	Custom simulator / Entirely evaluated / ∅	3MiB (262.1k entries)
InnerSP [6]	Not needed	Custom simulator / Entirely evaluated / ∅	≤976.5KiB (119.9k entries)
SortCache [84]	*N/A*	Custom simulator / Entirely evaluated / Single threaded, in-order core with 3 level cache	16.3MiB (1.4M entries)
ASA [100]	*N/A*	Modified *ZSim* / Entirely evaluated, one ASA per core / 8-cores with 3 level cache	8×8KiB (8×1k entries)
Flexagon [65]	Not needed	Custom simulator based on STONNE / Entirely evaluated / ∅	1.3MiB (327.7k entries)
GSCSp [57]	A, $B \rightarrow$ *row-major*	HLS / FPGA Prototype / ∅	2.1MiB (178.2k entries)

[1]We assume matrices are initially stored in *(UOP, CP) column-major* format (*CSC*).
[2]Only for *processor assist* accelerators.

The only accelerators that have a comparable baseline are the *standalone* ones addressing SpMSpM. They all compare themselves to OuterSPACE with the same benchmark matrices, so we can sort them by their speedup: Gamma ($6.6\times$)[10], InnerSP ($4.57\times$), SpArch ($4\times$), MatRaptor ($1.8\times$) and OuterSPACE ($1\times$). Three of the five *standalone* accelerators use the **Gust** algorithm (Gamma, InnerSP and MatRaptor). SpArch, an **OP** accelerator employing a complex architecture, outperforms MatRaptor, but does not achieve the same performance as InnerSP or Gamma, which suggests that **Gust** is most adapted to high-performance hardware accelerators.

These measurements are consistent with our model presented in Section 1.5.4 (Figure 1.6). OuterSPACE, the baseline, corresponds to graph ⑤. MatRaptor, the first **Gust** accelerator, was able to achieve higher performance by reducing the number of *random* accesses, and creating better regularity (red to yellow), as seen in graph ⑥.

SpArch, is an **OP** accelerator with a much larger local memory, which is used to address the red region in graph ⑤. Their A-matrix condensing method, allows them to reduce the number of *POs*, and approach the performance achievable in graph ⑧, although, *in fine*, it is impossible to store all the *POs* in local memory. InnerSP and Gamma correspond to optimizations of the **Gust** approach in graph ⑥, to approach the performance in graph ⑨. This is achieved by custom caches for the B-matrix to address the orange region. With the **Gust** algorithm, it is only necessary to store a limited number of B-rows, which is achievable with the large local memories in these accelerators. The model presented in Section 1.5.4 is helpful to understand the local memory requirements for SpMSpM accelerators.

2.3.2 Analysis for Designers

There is no single "best overall" accelerator. A designer must consider the required performance, the available area and the class of problems to be addressed. In Figure 2.12 we present a flow diagram which can guide a designer towards an architecture based on their requirements. Starting from the top-right of the figure, the designer must first decide between a *specialized accelerator* for a specific kernel or a *general purpose accelerator*. The former is preferable if performance must be maximized, requiring a *standalone* accelerator addressing both *gather* and *scatter* (*e.g.* MatRaptor, SpArch, Gamma and InnerSP).

[10]This speedup increases to $7.7\times$ with their proposed preprocessing.

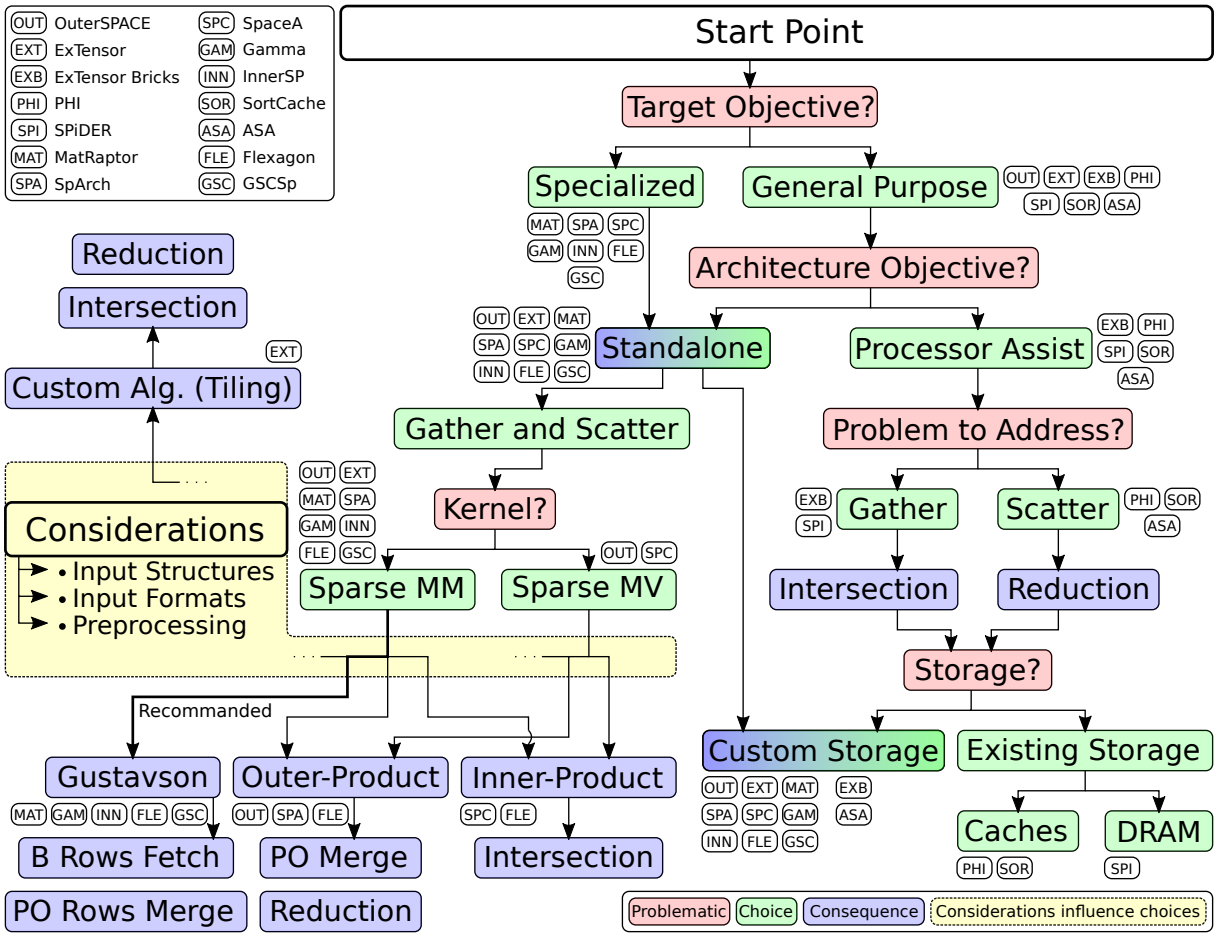


Figure 2.12: Flow Diagram for Architecture Selection

A *general purpose accelerator* can either take the form of a highly configurable *standalone* block (*e.g.* OuterSPACE and ExTensor) or a *processor assist* that boosts performance for a specific task. With a *processor assist* (rightmost branch in the figure), the processor still performs important parts of the kernel, typically the multiplications, but off-loads the *scatter* or *gather* tasks to the accelerator, as these are the tasks which are the bottleneck with a traditional cache hierarchy. For example, for an **IP** algorithm, the difficult part of the *gather* task is performing the *intersection* (e.g. the ExTensor bricks and SPiDRE). Conversely, with **OP** or **Gust**, the challenge with the *scatter* is to efficiently perform *reductions* (e.g. PHI, SortCache and ASA). Finally, the designer must decide whether the storage is done by reusing existing memory resources (*e.g.* PHI, SPiDRE and SortCache) or through custom caches/memories.

Going back to the case of a *standalone* accelerator, the algorithms differ according to the supported kernels. For sparse MV, the accelerators studied here have opted for an **OP** algorithm. For sparse MM, the designers of three of the five highest performing accelerators have chosen **Gust** algorithm, as this algorithm provides a good trade-off, simplifying the *gather* by reusing input B-rows and the row-by-row generation of the output, simplifying the *scatter* aspect. The fact that the outputs are generated row-by-row, facilitates parallel implementations. **Gust** works well with *(UOP, CP)*, providing consistent input and output formats. Beyond the basic algorithm, *tiling* approaches, as proposed by ExTensor, can be explored.

Orthogonal to previous choices, other design considerations are important. First, the starting point needs to be defined: some accelerators require software to perform *tiling* (*e.g.* ExTensor) or to rearrange the rows to improve performance (*e.g.* Gamma). The cost of such preprocessing can be high, and is usually only justified if the same matrix is reused multiple times.

2.4 Conclusion

The size of the datasets considered in computational physics, machine learning and graph analytics is continuing to grow and over the last six years, we note the emergence of many

hardware accelerators designed to improve upon the performance of CPUs and GPUs. In this paper, we have presented the most important designs and established a comparison, while highlighting the underlying hardware challenges associated with sparse multiplication kernels.

As we have seen, with this class of problem, the challenge is to efficiently use the memory system despite the dereferencing required to access the *metadata* associated with the *sparse data formats*. Depending on the selected MM algorithm, the major challenge is either with the *gather* and *intersection* or with the *scatter* updates. We have identified that accelerators based on *Gustavson's* algorithm achieve a trade-off, where, on the *gather* side, a limited number of B-rows need to be accessed, and on the output side, the result is generated row-by-row, simplifying the *scatter* updates. The best *Gustavson* accelerators have implemented further optimizations such as smart cache eviction strategies where the victim is selected by looking ahead at the A-indices. Several of the smaller *processor assist* accelerators reuse existing cache memories and simply modify the behaviour of the control logic (directories, tags, eviction policies) to optimize the operation for sparse matrices. On the other hand, the highest performance accelerators propose dedicated, massively parallel *reduction/mergers*, to maximize parallelism.

Given the number and diversity of existing designs, it is clear that further hardware optimizations and improvements are possible. As stated earlier, there may never be a single "best" sparse accelerator, but rather designs that are best suited to specific classes of problems. We believe that the structure of the matrices is critical in terms of the sizing of caches and other resources, and that in the coming years, new accelerators will be increasingly tailored based on the structure of the matrices for specific classes of problems.

Chapter 3

SpDCache

Chapter 4

Theory

Chapter 5

Microarchitecture

Chapter 6

Evaluation

Chapter 7

Optimisations

Conclusion

Contents

1 TODO

TODO

Appendices

Contents

A Sparse Formats

TODO

COO The simplest *sparse format* is called *COO* (*COOrdinate*) [20, 76, 89]. It consists in three separate tables, one filled only with the non-zero elements of the matrix, a row index table containing the row of the corresponding elements, and similarly a column index table (see Figure **??**). The two latter are *metadata* tables and all three tables are of size *nnz*. In this format, the elements are not necessarily sorted, the memory footprint does not depend on the structure of the matrix and all elements are stored independently of each other.

CSR & CSC The most commonly used *sparse format* is called *CSR* (*Compressed Sparse Row*) [11, 20, 76, 89]. It is similar to the *COO* format except that the row index table is replaced by a set of indices that point to the position of the start of each row in the column index table (see Figure **??**). The size of this table is the number of rows plus one, and, in practical cases, the memory footprint is smaller than with *COO*. However, the non-zero elements need to be sorted in *row-wise* order. This format has a *column-wise* variant called *CSC* (*Compressed Sparse Column*) where the column index table stores the positions of columns instead of rows. These two formats are symmetrical but with *CSR*, it is the traversal of rows that is efficient because they are sequential whereas in *CSC*, column traversal is efficient.

BCSR Even in sparse matrices, there can be regions, or blocks, that have a large fraction of non-zero values. In this case, the *BCSR* format [11], which is an extension of the *CSR* format with *tiling*, can be attractive. We use the term *tiling* to describe a level of hierarchy in a storage format. That is, a *tile* refers to a sub-matrix, which itself may be dense, as in the case of *BCSR*, or which could be sparse. With the *BCSR* format, we simply replace the non-zero elements of the *CSR* format with dense, fixed-size sub-matrices (see Figure **??**). Within these sub-matrices, it is possible to have some zero elements. The advantage of the *BCSR* format is that the dense sub-matrices can be processed using techniques optimized for dense matrices, particularly using vector units or GPUs [42]. Compared to the *CSR* format, *BCSR* makes it possible to accelerate the computation within the dense *tiles*, but this comes with the overhead of storing some zero values. Not all matrices are well suited to the *BCSR* format.

DIA For diagonally structured matrices ($\forall (i,j) \in [\![0,M]\!] \times [\![0,N]\!],\ a_{i,j} = 0\ for\ |i-j| > K$), a format called *DIA* (*DIAgonal*) is particularly well suited [11, 20, 76]. It consists in storing the values along the $2 \times K + 1$ diagonals in a table and keeping information about the position of the diagonals in the matrix (see Figure **??**). The organization of the non-zero data and selection of the diagonals can be tailored to the matrices and access patterns. This format is well suited to diagonal matrices, has a low overhead for storing indices and can be combined with other formats to better fit more complex structures.

ELL Another format called *ELL* (*ELLPACK*) [11, 20, 49, 76] is well suited when the number of non-zero elements per row can be bounded by K. In this case, the non-zero values in a $M \times N$ matrix can be stored as a dense $M \times K$ matrix, augmented with an index table, of the same dimensions, which gives the column position of each non-zero value (see Figure **??**). This format is not efficient if the number of non-zero elements per row is highly variable, as many entries in the $M \times K$ table are unused. This format is well suited to GPUs [18, 91] since it transforms a sparse matrix into a dense one in memory.

HYB The *HYB* format (HYBrid) [13] is an example of a format which combines *ELL* and *COO*. The *ELL* format can be used, but with a value of K (the width of the value table) that is smaller than the maximum non-zero entries per row in the original matrix (see Figure **??**). For the small number of rows where the table lacks a sufficient number of columns, the *COO* format can be used to store the extra values. In this way, the storage efficiency of *ELL* is improved, and it can be applied to a broader class of matrices.

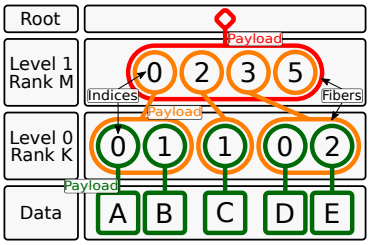


Figure 7.1: Example of the *FiberTree* Representation with CSR Format

B A Generic Representation for *Sparse Formats*

A visual representation of compressed matrices, called *FiberTree* [88], is helpful to understand these sparse formats. Each *rank* of a matrix represents a level of a tree where the nodes are filled with either the metadata of the child nodes or the matrix data which can only appear in the leaves (see Figure 7.1). With this representation, we clearly identify the number of *ranks*, which define the *fibers*. A *fiber* is the generic term to describe a one-dimensional stream of data or metadata, which corresponds to a row or column in the case of a two-dimensional matrix. A stream of indices is a *fiber* of metadata.Figure 7.1 presents, as an example, a *FiberTree* representation of the matrix of Figure **??**, matching the CSR format: each *rank* in the tree corresponds to a *fiber* stored in the table *Row PTR* or *Col IDX* of Figure 1.3b.

In a recent work [95], a systematic nomenclature for *sparse matrix (and tensor) formats* was proposed, based on *FiberTree* representation. In fact, in the example of Figure 7.1, the *fibers* are compressed with different methods. Thus each *rank* can be characterized by a label indicating how it is compressed: *Uncompressed* (*U*), *Coordinate Payload* (*CP*), *Uncompressed Offset Pairs* (*UOP*). *U* defines *ranks* where all data values are stored contiguously in memory. *CP* defines *ranks* that store the indices of each payload, a payload being either a non-zero element or a sub-matrix[1]. *UOP* defines *ranks* that store pointers (offsets) to delimit sub-matrices of the next *rank*[2]. These are the three main ones as they are used in the most common sparse formats but many others exist or can be created.

With this nomenclature, the sparse formats can be classified according to their *rank* compression methods. For example, the CSR format is denoted as *(UOP, CP) row-major* since its *Row PTR* table stores pointers (offsets) to rows. The CSR and CSC formats are both of type *(UOP, CP)*, one being *row-major*, the other *column-major*. The COO format is denoted as CP^2 because there are two coordinates (row and column) for each non-zero element, as it is a *CP fiber* having two indices per data element instead of one. User defined formats can also be easily described and named. For example, a user could define a storage format with a nested CSR. That is, each non-zero element in the outer CSR, corresponds to a sub-matrix, itself stored in CSR format. Such a representation would be labeled *(UOP, CP, UOP, CP)*. A similar nomenclature is proposed by TACO [20]. It is more adapted to code generation applications, where the *FiberTree*-based nomenclature better describes the memory storage of the sparse format. For example, *U* corresponds to *Dense*, *CP* to *Singleton*, the pair *(UOP, CP)* is translated as *Compressed* while *UOP* alone corresponds to *Offset*. We chose to adopt the *FiberTree*-based nomenclature in this article since it is more suited to a hardware context.

[1] In this section, we use the term sub-matrix but this is extended to the notion of *tiling* in Section 1.4.1.
[2] If some sub-matrices are empty, pointers will be repeated, hence 'uncompressed'.

C Extended Overview of the State-of-the-Art Accelerators for Sparse Computing

TODO

D Analytical Model of a *Reduction Cache* Computing an `OP` SpMV Kernel

TODO

E Python Model of a Data-Cache with Support for *Reductions*

TODO

F Statistics

TODO (get infos before end of contract)
(commits, nb. of lines, bugs, timeline -> into figures?)

List of Figures

List of Tables

List of Algorithms

List of Source Code

Bibliography

[1] James Alfred Ang, Brian W. Barrett, Kyle Bruce Wheeler, and Richard C Murphy. 2010. *Introducing the graph 500.* Technical Report. United States.

[2] Bahar Asgari, Ramyad Hadidi, Tushar Krishna, Hyesoon Kim, and Sudhakar Yalamanchili. 2020. ALRESCHA: A Lightweight Reconfigurable Sparse-Computation Accelerator. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 249–260. https://doi.org/10.1109/HPCA47549.2020.00029

[3] Venkitesh Ayyar, Evan Weinberg, Richard C. Brower, M.A. Clark, and Mathias Wagner. 2023. Optimizing Staggered Multigrid for Exascale performance. In *Proceedings of The 39th International Symposium on Lattice Field Theory — PoS(LATTICE2022)*, Vol. 430. 335. https://doi.org/10.22323/1.430.0335

[4] Ariful Azad, Aydin Buluç, and John Gilbert. 2015. Parallel Triangle Counting and Enumeration Using Matrix Algebra. In *2015 IEEE International Parallel and Distributed Processing Symposium Workshop*. 804–811. https://doi.org/10.1109/IPDPSW.2015.75

[5] Ariful Azad, Georgios A Pavlopoulos, Christos A Ouzounis, Nikos C Kyrpides, and Aydin Buluç. 2018. HipMCL: a high-performance parallel implementation of the Markov clustering algorithm for large-scale networks. *Nucleic Acids Research* 46, 6 (01 2018), e33–e33. https://doi.org/10.1093/nar/gkx1313 arXiv:https://academic.oup.com/nar/article-pdf/46/6/e33/24525991/gkx1313.pdf

[6] Daehyeon Baek, Soojin Hwang, Taekyung Heo, Daehoon Kim, and Jaehyuk Huh. 2021. InnerSP: A Memory Efficient Sparse Matrix Multiplication Accelerator with Locality-Aware Inner Product Processing. In *2021 30th International Conference on Parallel Architectures and Compilation Techniques (PACT)*. 116–128. https://doi.org/10.1109/PACT52795.2021.00016

[7] Ricardo Baeza-Yates and Alejandro Salinger. 2010. *Fast Intersection Algorithms for Sorted Sequences*. Springer Berlin Heidelberg, Berlin, Heidelberg, 45–61. https://doi.org/10.1007/978-3-642-12476-1_3

[8] Vignesh Balaji, Neal Crago, Aamer Jaleel, and Brandon Lucia. 2021. P-OPT: Practical Optimal Cache Replacement for Graph Analytics. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 668–681. https://doi.org/10.1109/HPCA51647.2021.00062

[9] Rajeev Balasubramonian, Andrew B. Kahng, Naveen Muralimanohar, Ali Shafiee, and Vaishnav Srinivas. 2017. CACTI 7: New Tools for Interconnect Exploration in Innovative Off-Chip Memories. *ACM Trans. Archit. Code Optim.* 14, 2, Article 14 (jun 2017), 25 pages. https://doi.org/10.1145/3085572

[10] Adrián Barredo, Jonathan C. Beard, and Miquel Moretó. 2019. POSTER: SPiDRE: Accelerating Sparse Memory Access Patterns. In *2019 28th International Conference on Parallel Architectures and Compilation Techniques (PACT)*. Institute of Electrical and Electronics Engineers, New York, NY, USA, 483–484. https://doi.org/10.1109/PACT.2019.00056

[11] Richard Barrett, Michael Berry, Tony F. Chan, James Demmel, June Donato, Jack Dongarra, Victor Eijkhout, Roldan Pozo, Charles Romine, and Henk van der Vorst. 1994. *Templates for the Solution of Linear Systems: Building Blocks for Iterative Methods*. Society for Industrial and Applied Mathematics. https://doi.org/10.1137/1.9781611971538

[12] Scott Beamer, Krste Asanović, and David Patterson. 2017. The GAP Benchmark Suite. (2017). https://doi.org/10.48550/arXiv.1508.03619 arXiv:1508.03619 [cs.DC]

[13] Nathan Bell and Michael Garland. 2009. Implementing Sparse Matrix-Vector Multiplication on Throughput-Oriented Processors. In *Proceedings of the Conference on High Performance Computing Networking, Storage and Analysis* (Portland, Oregon) *(SC '09)*. Association for Computing Machinery, New York, NY, USA, Article 18, 11 pages. https://doi.org/10.1145/1654059.1654078

[14] Achi Brandt and Dorit Ron. 2003. *Multigrid Solvers and Multilevel Optimization Strategies*. Springer US, Boston, MA, 1–69. https://doi.org/10.1007/978-1-4757-3748-6_1

[15] Sergey Brin and Lawrence Page. 1998. The anatomy of a large-scale hypertextual Web search engine. *Computer Networks and ISDN Systems* 30, 1 (1998), 107–117. https://doi.org/10.1016/S0169-7552(98)00110-X Proceedings of the Seventh International World Wide Web Conference.

[16] Benjamin Brock, Aydın Buluç, Timothy Mattson, Scott McMillan, and José Moreira. 2021. *The GraphBLAS C API Specification*. Technical Report. https://graphblas.org/

[17] Aydın Buluç, John Gilbert, and Viral B. Shah. 2011. *13. Implementing Sparse Matrices for Graph Algorithms*. Society for Industrial and Applied Mathematics, Chapter 13, 287–313. https://doi.org/10.1137/1.9780898719918.ch13

[18] Wei Cao, Lu Yao, Zongzhe Li, Yongxian Wang, and Zhenghua Wang. 2010. Implementing Sparse Matrix-Vector multiplication using CUDA based on a hybrid sparse matrix format. In *2010 International Conference on Computer Application and System Modeling (ICCASM 2010)*, Vol. 11. V11–161–V11–165. https://doi.org/10.1109/ICCASM.2010.5623237

[19] Timothy M. Chan. 2007. More Algorithms for All-Pairs Shortest Paths in Weighted Graphs. In *Proceedings of the Thirty-Ninth Annual ACM Symposium on Theory of Computing* (San Diego, California, USA) *(STOC '07)*. Association for Computing Machinery, New York, NY, USA, 590–598. https://doi.org/10.1145/1250790.1250877

[20] Stephen Chou, Fredrik Kjolstad, and Saman Amarasinghe. 2018. Format Abstraction for Sparse Tensor Algebra Compilers. *Proc. ACM Program. Lang.* 2, OOPSLA, Article 123 (oct 2018), 30 pages. https://doi.org/10.1145/3276493

[21] Xilinx Corporation. 2021. *Vitis Sparse Library*. Technical Report. https://xilinx.github.io/Vitis_Libraries/sparse/2021.1/index.html

[22] Vidushi Dadu, Jian Weng, Sihao Liu, and Tony Nowatzki. 2019. Towards General Purpose Acceleration by Exploiting Common Data-Dependence Forms. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture* (Columbus, OH, USA) *(MICRO '52)*. Association for Computing Machinery, New York, NY, USA, 924–939. https://doi.org/10.1145/3352460.3358276

[23] Steven Dalton, Nathan Bell, Luke Olson, and Michael Garland. 2014. Cusp: Generic Parallel Algorithms for Sparse Matrix and Graph Computations. Retrieved May 2022 from http://cusplibrary.github.io/ Version 0.5.1.

[24] Mehmet Deveci, Simon David Hammond, Michael M. Wolf, and Sivasankaran Rajamanickam. 2018. *Sparse Matrix-Matrix Multiplication on Multilevel Memory Architectures: Algorithms and Experiments*. Technical Report. United States. https://doi.org/10.2172/1435688

[25] Stijn Dongen. 2000. Graph Clustering by Flow Simulation. *PhD thesis, Center for Math and Computer Science (CWI)* (05 2000).

[26] Yixiao Du, Yuwei Hu, Zhongchun Zhou, and Zhiru Zhang. 2022. High-Performance Sparse Linear Algebra on HBM-Equipped FPGAs Using HLS: A Case Study on SpMV. In *Proceedings of the 2022 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays* (Virtual Event, USA) *(FPGA '22)*. Association for Computing Machinery, New York, NY, USA, 54–64. https://doi.org/10.1145/3490422.3502368

[27] Iain S. Duff, Michael A. Heroux, and Roldan Pozo. 2002. An Overview of the Sparse Basic Linear Algebra Subprograms: The New Standard from the BLAS Technical Forum. *ACM Trans. Math. Softw.* 28, 2 (jun 2002), 239–267. https://doi.org/10.1145/567806.567810

[28] D. K. Faddeev and V. N. Faddeeva. 1981. Computational methods of linear algebra. *Journal of Soviet Mathematics* 15, 5 (1981), 531–650. https://doi.org/10.1007/BF01086544

[29] Siying Feng, Jiawen Sun, Subhankar Pal, Xin He, Kuba Kaszyk, Dong-hyeon Park, Magnus Morton, Trevor Mudge, Murray Cole, Michael O'Boyle, Chaitali Chakrabarti, and Ronald Dreslinski. 2021. CoSPARSE: A Software and Hardware Reconfigurable SpMV Framework for Graph Analytics. In *2021 58th ACM/IEEE Design Automation Conference (DAC)*. 949–954. https://doi.org/10.1109/DAC18074.2021.9586114

[30] Trevor Gale, Matei Zaharia, Cliff Young, and Erich Elsen. 2020. Sparse GPU Kernels for Deep Learning. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis* (Atlanta, Georgia) *(SC '20)*. IEEE Press, Article 17, 14 pages. https://dl.acm.org/doi/10.5555/3433701.3433723

[31] Jianhua Gao, Weixing Ji, Fangli Chang, Shiyu Han, Bingxin Wei, Zeming Liu, and Yizhuo Wang. 2023. A Systematic Survey of General Sparse Matrix-Matrix Multiplication. *ACM Comput. Surv.* 55, 12, Article 244 (mar 2023), 36 pages. https://doi.org/10.1145/3571157

[32] Yingxue Gao, Lei Gong, Chao Wang, Teng Wang, Xi Li, and Xuehai Zhou. 2023. Algorithm/Hardware Co-Optimization for Sparsity-Aware SpMM Acceleration of GNNs. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 42, 12 (2023), 4763–4776. https://doi.org/10.1109/TCAD.2023.3281714

[33] John R. Gilbert, Steve Reinhardt, and Viral B. Shah. 2006. High-Performance Graph Algorithms from Parallel Sparse Matrices. In *Proceedings of the 8th International Conference on Applied Parallel Computing: State of the Art in Scientific Computing* (Umeå, Sweden) *(PARA'06)*. Springer-Verlag, Berlin, Heidelberg, 260–269. https://link.springer.com/chapter/10.1007/978-3-540-75755-9_32

[34] John R. Gilbert, Steve Reinhardt, and Viral B. Shah. 2008. A Unified Framework for Numerical and Combinatorial Computing. *Computing in Science & Engineering* 10, 2 (March 2008), 20–25. https://doi.org/10.1109/MCSE.2008.45

[35] Branko Grünbaum and Geoffrey C. Shephard. 1987. *Tilings and Patterns*. W. H. Freeman & Co., New York.

[36] Sumanth Gudaparthi, Sarabjeet Singh, Surya Narayanan, Rajeev Balasubramonian, and Visvesh Sathe. 2022. CANDLES: Channel-Aware Novel Dataflow-Microarchitecture Co-Design for Low Energy Sparse Neural Network Acceleration. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 876–891. https://doi.org/10.1109/HPCA53966.2022.00069

[37] Fred G. Gustavson. 1978. Two Fast Algorithms for Sparse Matrices: Multiplication and Permuted Transposition. *ACM Trans. Math. Softw.* 4, 3 (sep 1978), 250–269. https://doi.org/10.1145/355791.355796

[38] Song Han, Xingyu Liu, Huizi Mao, Jing Pu, Ardavan Pedram, Mark A. Horowitz, and William J. Dally. 2016. EIE: Efficient Inference Engine on Compressed Deep Neural Network. *SIGARCH Comput. Archit. News* 44, 3 (jun 2016), 243–254. https://doi.org/10.1145/3007787.3001163

[39] Václav Hapla, David Horák, and Michal Merta. 2012. Use of Direct Solvers in TFETI Massively Parallel Implementation. In *Proceedings of the 11th International Conference on Applied Parallel and Scientific Computing* (Helsinki, Finland) *(PARA'12)*. Springer-Verlag, Berlin, Heidelberg, 192–205. https://doi.org/10.1007/978-3-642-36803-5_14

[40] Kartik Hegde, Hadi Asghari-Moghaddam, Michael Pellauer, Neal Crago, Aamer Jaleel, Edgar Solomonik, Joel Emer, and Christopher W. Fletcher. 2019. ExTensor: An Accelerator for Sparse Tensor Algebra. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture* (Columbus, OH, USA) *(MICRO '52)*. Association for Computing Machinery, New York, NY, USA, 319–333. https://doi.org/10.1145/3352460.3358275

[41] Yuwei Hu, Yixiao Du, Ecenur Ustun, and Zhiru Zhang. 2021. GraphLily: Accelerating Graph Linear Algebra on HBM-Equipped FPGAs. In *2021 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. 1–9. https://doi.org/10.1109/ICCAD51958.2021.9643582

[42] Eun-Jin Im, Katherine Yelick, and Richard Vuduc. 2004. Sparsity: Optimization Framework for Sparse Matrix Kernels. *The International Journal of High Performance Computing Applications* 18, 1 (2004), 135–158. https://doi.org/10.1177/1094342004041296 arXiv:https://doi.org/10.1177/1094342004041296

[43] Intel. 2003. Intel Math Kernel Library. Retrieved May 2022 from https://www.intel.com/content/www/us/en/developer/tools/oneapi/onemkl.html#gs.ylo2j7

[44] Valentin Isaac--Chassande, Adrian Evans, Yves Durand, and Frédéric Rousseau. 2024. Dedicated Hardware Accelerators for Processing of Sparse Matrices and Vectors: A Survey. *ACM Trans. Archit. Code Optim.* 21, 2, Article 27 (Feb. 2024), 26 pages. https://doi.org/10.1145/3640542

[45] Valentin Isaac--Chassande, Adrian Evans, Yves Durand, and Frédéric Rousseau. 2024. SpDCache: Region-Based Reduction Cache for Outer-Product Sparse Matrix Kernels. In *2024 IEEE 35th International Conference on Application-specific Systems, Architectures and Processors (ASAP)*. IEEE Computer Society, Los Alamitos, CA, USA, 3–7. https://doi.org/10.1109/ASAP61560.2024.00012

[46] Satoshi Itoh, Pablo Ordejón, and Richard M. Martin. 1995. Order-N tight-binding molecular dynamics on parallel computers. *Computer Physics Communications* 88, 2 (1995), 173–185. https://doi.org/10.1016/0010-4655(95)00031-A

[47] JESD235A 2016. *JEDEC Updates Groundbreaking High Bandwidth Memory (HBM) Standard*. Technical Report. JEDEC. https://www.jedec.org/news/pressreleases/jedec-updates-groundbreaking-high-bandwidth-memory-hbm-standard

[48] Jeremy Kepner, David Bader, Aydın Buluç, John Gilbert, Timothy Mattson, and Henning Meyerhenke. 2015. Graphs, Matrices, and the GraphBLAS: Seven Good Reasons. *Procedia Computer Science* 51, International Conference On Computational Science, ICCS 2015 (2015), 2453–2462. https://doi.org/10.1016/j.procs.2015.05.353

[49] David R. Kincaid, Thomas C. Oppe, and David M. Young. 1989. *ITPACKV 2D user's guide*. Technical Report. United States. https://doi.org/10.2172/7093021

[50] Vladimir Kiriansky, Yunming Zhang, and Saman Amarasinghe. 2016. Optimizing Indirect Memory References with Milk. In *Proceedings of the 2016 International Conference on Parallel Architectures and Compilation* (Haifa, Israel) *(PACT '16)*. Association for Computing Machinery, New York, NY, USA, 299–312. https://doi.org/10.1145/2967938.2967948

[51] Fredrik Kjolstad, Shoaib Kamil, Stephen Chou, David Lugato, and Saman Amarasinghe. 2017. The Tensor Algebra Compiler. *Proc. ACM Program. Lang.* 1, OOPSLA, Article 77 (oct 2017), 29 pages. https://doi.org/10.1145/3133901

[52] Scott P. Kolodziej, Mohsen Aznaveh, Matthew Bullock, Jarrett David, Timothy A. Davis, Matthew Henderson, Yifan Hu, and Read Sandstrom. 2019. The SuiteSparse Matrix Collection Website Interface. *Journal of Open Source Software* 4, 35 (2019), 1244. https://doi.org/10.21105/joss.01244

[53] Monica D. Lam, Edward E. Rothberg, and Michael E. Wolf. 1991. The Cache Performance and Optimizations of Blocked Algorithms. *SIGPLAN Not.* 26, 4 (apr 1991), 63–74. https://doi.org/10.1145/106973.106981

[54] Jure Leskovec and Andrej Krevl. 2014. *SNAP Datasets: Stanford Large Network Dataset Collection*. Retrieved May 2022 from http://snap.stanford.edu/data

[55] Ruipeng Li, Björn Sjögreen, and Ulrike Meier Yang. 2021. A New Class of AMG Interpolation Methods Based on Matrix-Matrix Multiplications. *SIAM Journal on Scientific Computing* 43, 5 (2021), S540–S564. https://doi.org/10.1137/20M134931X

[56] Sheng Li, Jung Ho Ahn, Richard D. Strong, Jay B. Brockman, Dean M. Tullsen, and Norman P. Jouppi. 2009. McPAT: An integrated power, area, and timing modeling framework for multicore and manycore architectures. In *2009 42nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. Institute of Electrical and Electronics Engineers and Association for Computing Machinery, New York, NY, USA, 469–480.

[57] Shiqing Li, Shuo Huai, and Weichen Liu. 2023. An Efficient Gustavson-Based Sparse Matrix–Matrix Multiplication Accelerator on Embedded FPGAs. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 42, 12 (Dec 2023), 4671–4680. https://doi.org/10.1109/TCAD.2023.3281719

[58] Shiqing Li, Shuo Huai, and Weichen Liu. 2023. An Efficient Gustavson-Based Sparse Matrix–Matrix Multiplication Accelerator on Embedded FPGAs. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 42, 12 (Dec 2023), 4671–4680. https://doi.org/10.1109/TCAD.2023.3281719

[59] Weifeng Liu and Brian Vinter. 2014. An Efficient GPU General Sparse Matrix-Matrix Multiplication for Irregular Data. In *2014 IEEE 28th International Parallel and Distributed Processing Symposium*. 370–381. https://doi.org/10.1109/IPDPS.2014.47

[60] Jason Lowe-Power, Abdul Mutaal Ahmad, Ayaz Akram, Mohammad Alian, Rico Amslinger, Matteo Andreozzi, Adrià Armejach, Nils Asmussen, Brad Beckmann, Srikant Bharadwaj, Gabe Black, Gedare Bloom, Bobby R. Bruce, Daniel Rodrigues Carvalho, Jeronimo Castrillon, Lizhong Chen, Nicolas Derumigny, Stephan Diestelhorst, Wendy Elsasser, Carlos Escuin, Marjan Fariborz, Amin Farmahini-Farahani, Pouya Fotouhi, Ryan Gambord, Jayneel Gandhi, Dibakar Gope, Thomas Grass, Anthony Gutierrez, Bagus Hanindhito, Andreas Hansson, Swapnil Haria, Austin Harris, Timothy Hayes, Adrian Herrera, Matthew Horsnell, Syed Ali Raza Jafri, Radhika Jagtap, Hanhwi Jang, Reiley Jeyapaul, Timothy M. Jones, Matthias Jung, Subash Kannoth, Hamidreza Khaleghzadeh, Yuetsu Kodama, Tushar Krishna, Tommaso Marinelli, Christian Menard, Andrea Mondelli, Miquel Moreto, Tiago Mück, Omar Naji, Krishnendra Nathella, Hoa Nguyen, Nikos Nikoleris, Lena E. Olson, Marc Orr, Binh Pham, Pablo Prieto, Trivikram Reddy, Alec Roelke, Mahyar Samani, Andreas Sandberg, Javier Setoain, Boris Shingarov, Matthew D. Sinclair, Tuan Ta, Rahul Thakur, Giacomo Travaglini, Michael Upton, Nilay Vaish, Ilias Vougioukas, William Wang, Zhengrong Wang, Norbert Wehn, Christian Weis, David A. Wood, Hongil Yoon, and Éder F. Zulian. 2020. The gem5 Simulator: Version 20.0+. https://doi.org/10.48550/ARXIV.2007.03152

[61] J.-C. Luo. 1992. Algorithms for Reducing the Bandwidth and Profile of a Sparse Matrix. *Computers & Structures* 44, 3 (1992), 535–548. https://doi.org/10.1016/0045-7949(92)90386-E

[62] Liviu Mafteiu-Scai. 2014. The Bandwidths of a Matrix. A Survey of Algorithms. *West University of Timisoara Annals, Timisoara, Romania* LII (12 2014), 183– 223. https://doi.org/10.2478/awutm-2014-0019

[63] Kiran Matam, Siva Rama Krishna Bharadwaj Indarapu, and Kishore Kothapalli. 2012. Sparse matrix-matrix multiplication on modern architectures. In *2012 19th International Conference on High Performance Computing*. 1–10. https://doi.org/10.1109/HiPC.2012.6507483

[64] Thaha Mohammed and Rashid Mehmood. 2022. Performance Enhancement Strategies for Sparse Matrix-Vector Multiplication (SpMV) and Iterative Linear Solvers. (2022). https://doi.org/10.48550/ARXIV.2212.07490 arXiv:2212.07490 [cs.DS]

[65] Francisco Muñoz Martínez, Raveesh Garg, Michael Pellauer, José L. Abellán, Manuel E. Acacio, and Tushar Krishna. 2023. Flexagon: A Multi-Dataflow Sparse-Sparse Matrix Multiplication Accelerator for Efficient DNN Processing. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (Vancouver, BC, Canada) *(ASPLOS 2023)*. Association for Computing Machinery, New York, NY, USA, 252–265. https://doi.org/10.1145/3582016.3582069

[66] Anurag Mukkara, Nathan Beckmann, and Daniel Sanchez. 2019. PHI: Architectural Support for Synchronization- and Bandwidth-Efficient Commutative Scatter Updates. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture* (Columbus, OH, USA) *(MICRO '52)*. Association for Computing Machinery, New York, NY, USA, 1009–1022. https://doi.org/10.1145/3352460.3358254

[67] Francisco Muñoz-Martínez, José L. Abellán, Manuel E. Acacio, and Tushar Krishna. 2021. STONNE: Enabling Cycle-Level Microarchitectural Simulation for DNN Inference Accelerators. In *2021 IEEE International Symposium on Workload Characterization (IISWC)*. 201–213. https://doi.org/10.1109/IISWC53511.2021.00028

[68] Yusuke Nagasaka, Satoshi Matsuoka, Ariful Azad, and Aydın Buluç. 2018. High-Performance Sparse Matrix-Matrix Products on Intel KNL and Multicore Architectures. In *Workshop Proceedings of the 47th International Conference on Parallel Processing* (Eugene, OR, USA) *(ICPP Workshops '18)*. Association for Computing Machinery, New York, NY, USA, Article 34, 10 pages. https://doi.org/10.1145/3229710.3229720

[69] NVIDIA. 2014. cuSPARSE Library. Retrieved May 2022 from https://developer.nvidia.com/cusparse

[70] Subhankar Pal, Jonathan Beaumont, Dong-Hyeon Park, Aporva Amarnath, Siying Feng, Chaitali Chakrabarti, Hun-Seok Kim, David Blaauw, Trevor Mudge, and Ronald Dreslinski. 2018. OuterSPACE: An Outer Product Based Sparse Matrix Multiplication Accelerator. In *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. Institute of Electrical and Electronics Engineers, New York, NY, USA, 724–736. https://doi.org/10.1109/HPCA.2018.00067

[71] A. Parashar, M. Rhu, A. Mukkara, A. Puglielli, R. Venkatesan, B. Khailany, J. Emer, S. W. Keckler, and W. J. Dally. 2017. SCNN: An accelerator for compressed-sparse convolutional neural networks. In *2017 ACM/IEEE 44th Annual International Symposium on Computer Architecture (ISCA)*. IEEE Computer Society, Los Alamitos, CA, USA, 27–40. https://doi.org/10.1145/3079856.3080254

[72] Gerald Penn. 2006. Efficient transitive closure of sparse matrices over closed semirings. *Theoretical Computer Science* 354, 1 (2006), 72–81. https://doi.org/10.1016/j.tcs.2005.11.008 Algebraic Methods in Language Processing.

[73] Michael O Rabin and Vijay V Vazirani. 1989. Maximum matchings in general graphs through randomization. *Journal of Algorithms* 10, 4 (1989), 557–567. https://doi.org/10.1016/0196-6774(89)90005-9

[74] Vijay Janapa Reddi, Christine Cheng, David Kanter, Peter Mattson, Guenther Schmuelling, Carole-Jean Wu, Brian Anderson, Maximilien Breughe, Mark Charlebois, William Chou, Ramesh Chukka, Cody Coleman, Sam Davis, Pan Deng, Greg Diamos, Jared Duke, Dave Fick, J. Scott Gardner, Itay Hubara, Sachin Idgunji, Thomas B. Jablin, Jeff Jiao, Tom St. John, Pankaj Kanwar, David Lee, Jeffery Liao, Anton Lokhmotov, Francisco Massa, Peng Meng, Paulius Micikevicius, Colin Osborne, Gennady Pekhimenko, Arun Tejusve Raghunath Rajan, Dilip Sequeira, Ashish Sirasao, Fei Sun, Hanlin Tang, Michael Thomson, Frank Wei, Ephrem Wu, Lingjie Xu, Koichi Yamada, Bing Yu, George Yuan, Aaron Zhong, Peizhao Zhang, and Yuchen Zhou. 2020. MLPerf Inference Benchmark. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. 446–459. https://doi.org/10.1109/ISCA45697.2020.00045

[75] Oleg I. Ryabkov. 2022. Implementation of the Algebraic Multigrid Solver Designed for Graphics Processing Units Based on the AMGCL Framework. In *Parallel Computational Technologies*, Leonid Sokolinsky and Mikhail Zymbler (Eds.), Vol. 1618. Springer International Publishing, Cham, 131–142. https://doi.org/10.1007/978-3-031-11623-0_10

[76] Yousef Saad. 2003. *Iterative Methods for Sparse Linear Systems* (second ed.). Society for Industrial and Applied Mathematics. https://doi.org/10.1137/1.9780898718003

[77] Fazle Sadi, Joe Sweeney, Tze Meng Low, James C. Hoe, Larry Pileggi, and Franz Franchetti. 2019. Efficient SpMV Operation for Large and Highly Sparse Matrices Using Scalable Multi-Way Merge Parallelization. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture* (Columbus, OH, USA) *(MICRO '52)*. Association for Computing Machinery, New York, NY, USA, 347–358. https://doi.org/10.1145/3352460.3358330

[78] Daniel Sanchez and Christos Kozyrakis. 2013. ZSim: Fast and Accurate Microarchitectural Simulation of Thousand-Core Systems. In *Proceedings of the 40th Annual International Symposium on Computer Architecture* (Tel-Aviv, Israel) *(ISCA '13)*. Association for Computing Machinery, New York, NY, USA, 475–486. https://doi.org/10.1145/2485922.2485963

[79] Conrad Sanderson and Ryan Curtin. 2016. Armadillo: a template-based C++ library for linear algebra. *Journal of Open Source Software* 1, 2 (2016), 26. https://doi.org/10.21105/joss.00026

[80] P. Scheffler, F. Zaruba, F. Schuiki, T. Hoefler, and L. Benini. 2023. Sparse Stream Semantic Registers: A Lightweight ISA Extension Accelerating General Sparse Linear Algebra. *IEEE Transactions on Parallel and Distributed Systems* 34, 12 (dec 2023), 3147–3161. https://doi.org/10.1109/TPDS.2023.3322029

[81] Shaden Smith, Jee W. Choi, Jiajia Li, Richard Vuduc, Jongsoo Park, Xing Liu, and George Karypis. 2017. *FROSTT: The Formidable Repository of Open Sparse Tensors and Tools*. Retrieved May 2022 from http://frostt.io/

[82] Linghao Song, Youwei Zhuo, Xuehai Qian, Hai Li, and Yiran Chen. 2018. GraphR: Accelerating Graph Processing Using ReRAM. In *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 531–543. https://doi.org/10.1109/HPCA.2018.00052

[83] Sriseshan Srikanth, Thomas M. Conte, Erik P. DeBenedictis, and Jeanine Cook. 2017. The superstrider architecture: Integrating logic and memory towards non-von Neumann computing. *2017 IEEE International Conference on Rebooting Computing, ICRC 2017 - Proceedings* 2017-January (2017), 1 – 8. https://doi.org/10.1109/ICRC.2017.8123669

[84] Sriseshan Srikanth, Anirudh Jain, Thomas M. Conte, Erik P. Debenedictis, and Jeanine Cook. 2021. SortCache: Intelligent Cache Management for Accelerating Sparse Data Workloads. *ACM Trans. Archit. Code Optim.* 18, 4, Article 56 (sep 2021), 24 pages. https://doi.org/10.1145/3473332

[85] Sriseshan Srikanth, Anirudh Jain, Joseph M. Lennon, Thomas M. Conte, Erik Debenedictis, and Jeanine Cook. 2019. MetaStrider: Architectures for Scalable Memory-centric Reduction of Sparse Data Streams. *ACM Trans. Archit. Code Optim.* 16, 4, Article 35 (oct 2019), 26 pages. https://doi.org/10.1145/3355396

[86] Nitish Srivastava, Hanchen Jin, Jie Liu, David Albonesi, and Zhiru Zhang. 2020. MatRaptor: A Sparse-Sparse Matrix Multiplication Accelerator Based on Row-Wise Product. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. Institute of Electrical and Electronics Engineers, New York, NY, USA, 766–780. https://doi.org/10.1109/MICRO50266.2020.00068

[87] Nitish Srivastava, Hanchen Jin, Shaden Smith, Hongbo Rong, David Albonesi, and Zhiru Zhang. 2020. Tensaurus: A Versatile Accelerator for Mixed Sparse-Dense Tensor Computations. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 689–702. https://doi.org/10.1109/HPCA47549.2020.00062

[88] Vivienne Sze, Yu-Hsin Chen, Tien-Ju Yang, and Joel Emer. 2020. *Efficient Processing of Deep Neural Networks* (1 ed.). Springer Cham. https://doi.org/10.1007/978-3-031-01766-7

[89] Reginald P. Tewarson. 1973. *Sparse matrices*. Vol. 69. Academic press New York, State University of New York, Stony Brook, NY.

[90] Richard Vuduc, James W Demmel, and Katherine A Yelick. 2005. OSKI: A library of automatically tuned sparse matrix kernels. *Journal of Physics: Conference Series* 16, 1 (jan 2005), 521. https://doi.org/10.1088/1742-6596/16/1/071

[91] F. Vázquez, E M Garzon, Jose Martinez, and J Fernández. 2009. The sparse matrix vector product on GPUs. *Proceedings of the 2009 International Conference on Computational and Mathematical Methods in Science and Engineering* 2 (01 2009).

[92] Chen Wang, Chuan Xu, and Abdel Lisser. 2014. Bandwidth Minimization Problem. In *10th International Conference on MOdeling, Optimization and SIMulation - MOSIM14*. Nancy, France. https://hal.science/hal-01166658

[93] Lucas Wilkinson, Kazem Cheshmi, and Maryam Mehri Dehnavi. 2023. Register Tiling for Unstructured Sparsity in Neural Network Inference. *Proc. ACM Program. Lang.* 7, PLDI, Article 188 (June 2023), 26 pages. https://doi.org/10.1145/3591302

[94] Samuel Williams, Leonid Oliker, Richard Vuduc, John Shalf, Katherine Yelick, and James Demmel. 2007. Optimization of sparse matrix-vector multiplication on emerging multicore platforms. In *SC '07: Proceedings of the 2007 ACM/IEEE Conference on Supercomputing*. 1–12. https://doi.org/10.1145/1362622.1362674

[95] Yannan Nellie Wu, Po-An Tsai, Angshuman Parashar, Vivienne Sze, and Joel S. Emer. 2022. Sparseloop: An Analytical Approach To Sparse Tensor Accelerator Modeling. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. Institute of Electrical and Electronics Engineers and Association for Computing Machinery, New York, NY, USA, 1377–1395. https://doi.org/10.1109/MICRO56248.2022.00096

[96] Wm. A. Wulf and Sally A. McKee. 1995. Hitting the Memory Wall: Implications of the Obvious. *SIGARCH Comput. Archit. News* 23, 1 (mar 1995), 20–24. https://doi.org/10.1145/216585.216588

[97] Xinfeng Xie, Zheng Liang, Peng Gu, Abanti Basak, Lei Deng, Ling Liang, Xing Hu, and Yuan Xie. 2021. SpaceA: Sparse Matrix Vector Multiplication on Processing-in-Memory Accelerator. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 570–583. https://doi.org/10.1109/HPCA51647.2021.00055

[98] Ichitaro Yamazaki and Xiaoye S. Li. 2011. On Techniques to Improve Robustness and Scalability of a Parallel Hybrid Linear Solver. In *High Performance Computing for Computational Science – VECPAR 2010*, José M. Laginha M. Palma, Michel Daydé, Osni Marques, and João Correia Lopes (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 421–434.

[99] Orestis Zachariadis, Nitin Satpute, Juan Gómez-Luna, and Joaquín Olivares. 2020. Accelerating sparse matrix–matrix multiplication with GPU Tensor Cores. *Computers & Electrical Engineering* 88 (2020), 106848. https://doi.org/10.1016/j.compeleceng.2020.106848

[100] Chao Zhang, Maximilian Bremer, Cy Chan, John Shalf, and Xiaochen Guo. 2022. ASA: Accelerating Sparse Accumulation in Column-Wise SpGEMM. *ACM Trans. Archit. Code Optim.* 19, 4, Article 49 (sep 2022), 24 pages. https://doi.org/10.1145/3543068

[101] Guowei Zhang, Nithya Attaluri, Joel S. Emer, and Daniel Sanchez. 2021. Gamma: Leveraging Gustavson's Algorithm to Accelerate Sparse Matrix Multiplication. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems* (Virtual, USA) *(ASPLOS '21)*. Association for Computing Machinery, New York, NY, USA, 687–701. https://doi.org/10.1145/3445814.3446702

[102] Guowei Zhang, Webb Horn, and Daniel Sanchez. 2015. Exploiting commutativity to reduce the cost of updates to shared data in cache-coherent systems. In *2015 48th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. Institute of Electrical and Electronics Engineers and Association for Computing Machinery, New York, NY, USA, 13–25. https://doi.org/10.1145/2830772.2830774

[103] Guowei Zhang and Daniel Sanchez. 2019. Leveraging Caches to Accelerate Hash Tables and Memoization. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture* (Columbus, OH, USA) *(MICRO '52)*. Association for Computing Machinery, New York, NY, USA, 440–452. https://doi.org/10.1145/3352460.3358272

[104] Zhekai Zhang, Hanrui Wang, Song Han, and William J. Dally. 2020. SpArch: Efficient Architecture for Sparse Matrix Multiplication. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE Computer Society, Los Alamitos, CA, USA, 261–274. https://doi.org/10.1109/HPCA47549.2020.00030

Contents

Plan (Brouillon)

- Introduction **(~3 p.)**
  - "light" > give the entry point on sparse matrices and irregular accesses **(1-2 p.)**
  - Mains contributions **(1 p.)**
  - introduce the structure of the manuscript
- Background **(~5 p.)**
  - irregular accesses -> def, issues (in memory but not only?), domains/applications (including sparse matrix processing) **(1 p.)**
  - Focus on sparse matrices, context: it is widely used, in many domains, sparse, large, structures
  - Sparse matrices in a kernel: show where the accesses are regular vs irregular (infos on basic sparse formats (other sparse formats in appendix x), tiling, basic kernels, basic algos...) **(2-3 p.)**
  - Hardware challenges clearly defined (gather, scatter...) **(1 p.)**
- SoA **(~11 p.)**
  - Present solutions in SW for CPU/GPU, such as special sparse formats, possibility to play on algo (gust analysis), tiling methods adapted for sparse data (specific sparse format + specific algos) (expliquer clairement que ces solutions sont interressantes et pourquoi on se concentre sur le HW) **(1-2 p.)**
  - Present solutions in HW (dedicated accelerators) -> comparison table along common characteristics **(1-2 p.)**
  - Presentation of the major accelerators (selection of 3 important: processor assist PHI, standalone GAMMA, multi algo SPADA, ACES) (others in appendix xi) **(3-4 p.)**
  - Problem of comparison / comparison of standalone / other analysis (issue of matrix structure) -> Objectives: convince reader that SoA was done in depth, and acknoledge the matrice structure issue **(1 p.)**
  - Takeaways -> paths for designing an accelerator (end od SoA paper) **(1-2 p.)**
  - We choose a path -> show which one (coarse grain, at the level of the SoA, comparable to SortCache, PHI...)
- High level architectural presentation of SpDCache **(~10 p.)**
  - Reminder of the path we choose with more details: cache / reductions / OP SpMV / ... **(1 p.)**
  - Emphasis the comparison between concerned SoA accelerators -> show what is already done and useful, show what is missing
  - Present the case of banded matrices as an example of structure to leverage **(1-2 p.)**
  - SpDCache, architecture presentation with two regions, details on REDs transactions, IBRC vs OBRT **(7 p.)**
  - Initial results from Python model (ASAP) and note that a probabilistic model and RTL model will follow
- Mathematical analysis of the cache memory accesses for processing sparse matrices **(~23 p.)**
  - Explain the objectives of this chapter (~formal justification of the concept + cache size/region dimensioning) **(1 p.)**
  - Describe the design space of the path we choose (with reminders if necessary) -> introduce base parameters and/or variables, give the exact algorithm... **(1 p.)**
  - Show the interest of separate cache regions for A/b and c (formal) -> show that a GC is obviously not efficient when random accesses / confirm why SortCache, PHI, ... exist **(2 p.)**

    - Show that A and b are not a limiting factor with a formal reasoning -> give the needed space for GRC when prefetcher (clear statement that gather is not the aim of the thesis) **(2-3 p.)**
    - We use a GRC (no modification) for A and b -> get nb loads
    - Show the interest of a reduction cache region for c -> REDs / reduce the number of mem accesses / link to SoA (see ASAP) **(2 p.)**
    - We created a probabilistic model of the reduction cache which is based on the following principles ... (presented in detail in appendix n) / avantage of this model / we implemented this model in C / validation of the model with state of the art article on generic cache formalisation / introduce the graphs for the next sections **(1-2 p.)**
    - Show that reduction cache does not handle randomness -> model graph **(1-2 p.)**
    - Show RedC 1reg vs RedC 2reg on banded example **(2-3 p.)**
    - Show best cache dimensions for the dense part (band) **(1 p.)**
    - Show that sparse region (out of band) could use an other region for fine grain density (case of sub bands) -> dimensions **(1-2 p.)**
    - Show that sparse region could use adapted HW because of the isolated elements -> dimensions **(1-2 p.)**
    - Show how it would work with a uniform matrix -> how does it affects the dimensions **(1 p.)**
    - Conclude on our solution and dimensioning **(1 p.)**
- Micro architecture of SpDCache **(~10 p.)**
    - Present our hardware solution, based on the concept
    - Start with basic info on HPDcache (only the one that are useful for SpDCache)
    - Describe modif in pipeline / hazard management / throughput
    - Modif in RAM accesses ...
    - Stats: git / bugs / lines of code / synthesis
- RTL simulation results **(~10 p.)**
    - Experimentations: RTL simulation results -> matrices chosen, generated / metrics / ...
    - Results / comparison with theory / analysis / conclusion
- Future enhancement **(~8 p.)**
    - Architecture presentation with three regions (add OBRC, CB...) **(2 p.)**
    - Architecture of eviction anticipation **(2 p.)**
    - Present multicore version -> scale **(2 p.)**
    - Present additional optimisation (region config, learn from first iteration) **(1-2 p.)**
- Conclusions **(~2 p.)**
    - ..
- Appendices
    - All sparse formats **(~5 p.)**
    - Other accelerators of SoA **(~10 p.)**
    - Probabilistic model of the reduction cache **(~10 p.)**
    - Python model **(~1 p.)**